CSE4007
Artificial Intelligence

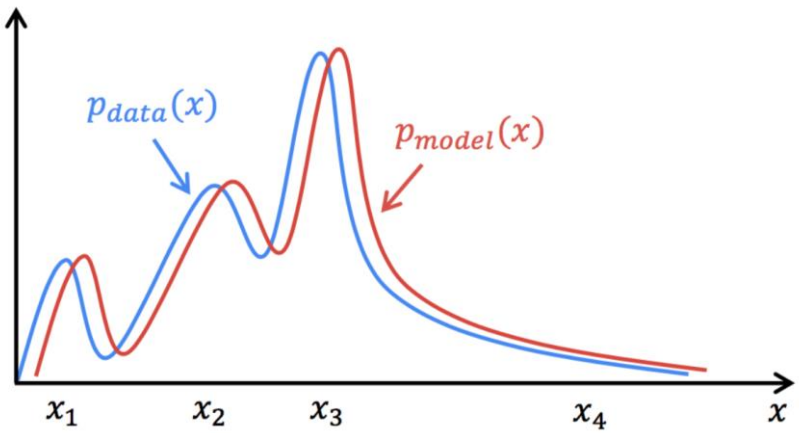
Lecture 12: Generative Models Part 4

Diffusion Models

Department of Data Science
Hanyang University

Generative Model

- The goal of the generative model is to find a $p_{model}(x)$ that a pproximates $p_{data}(x)$ well.

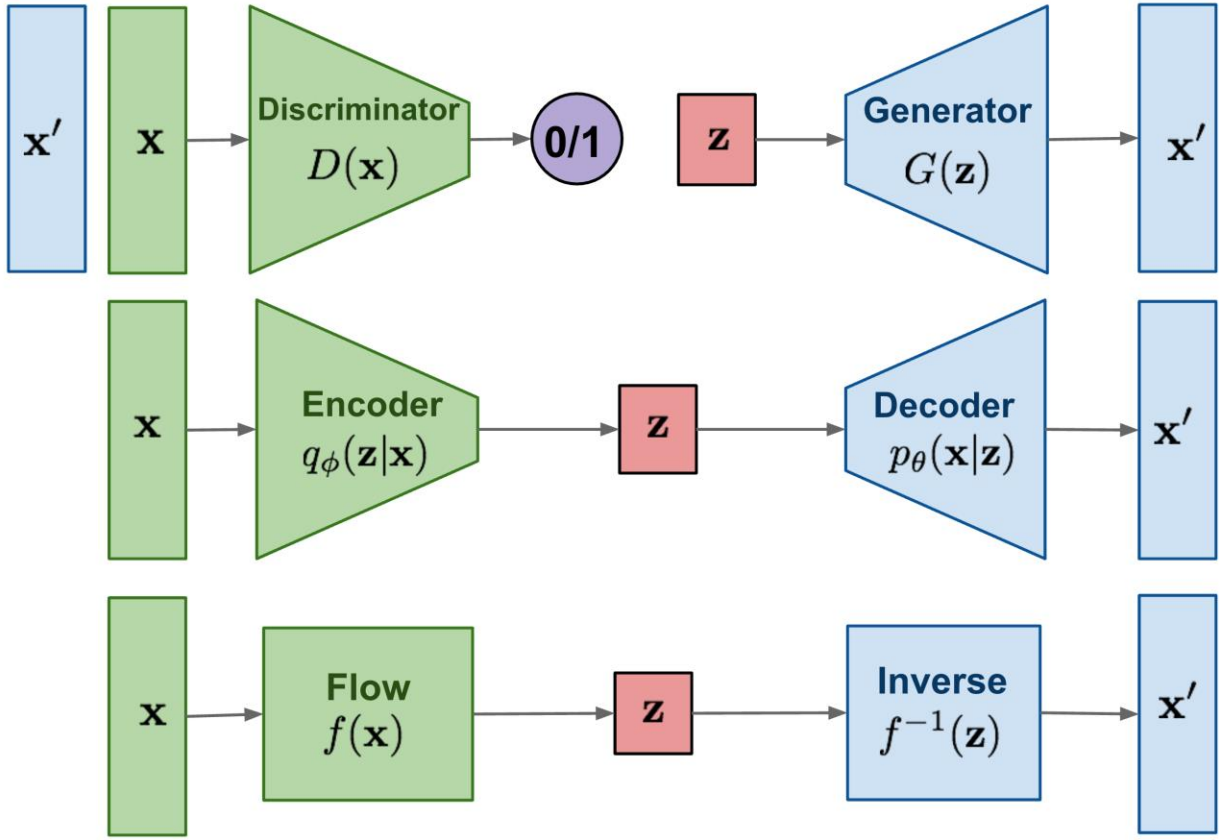


Generative Model: Normalizing Flow

GAN: Adversarial training

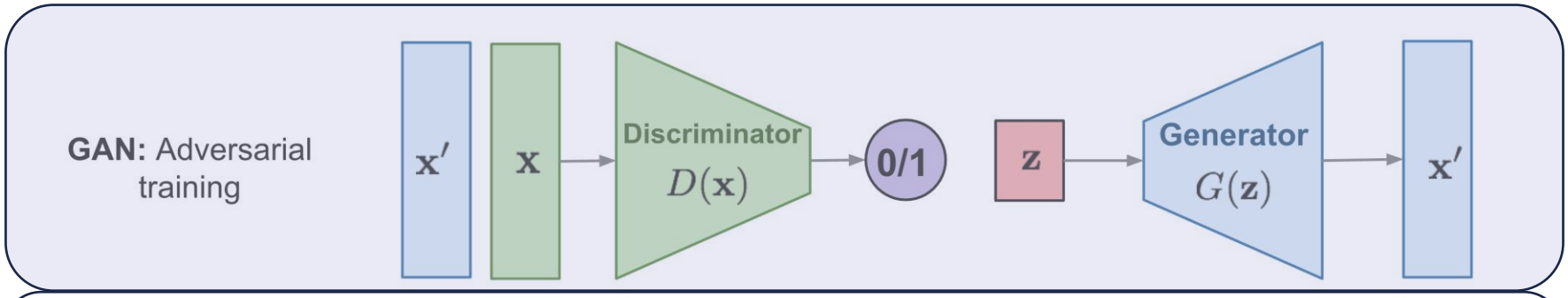
VAE: maximize variational lower bound

Flow-based models: Invertible transform of distributions



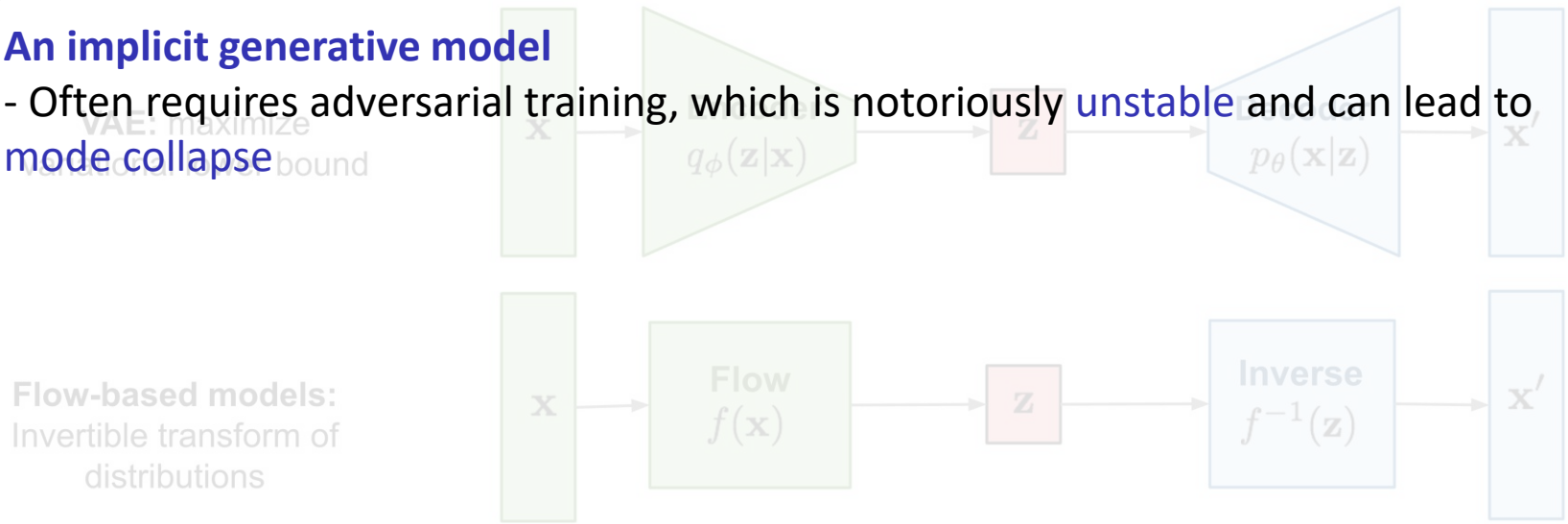
https://lilianweng.github.io/posts/2018-10-13-flow-models/

Generative Model: Normalizing Flow



An implicit generative model

- Often requires adversarial training, which is notoriously unstable and can lead to mode collapse



https://lilianweng.github.io/posts/2018-10-13-flow-models/

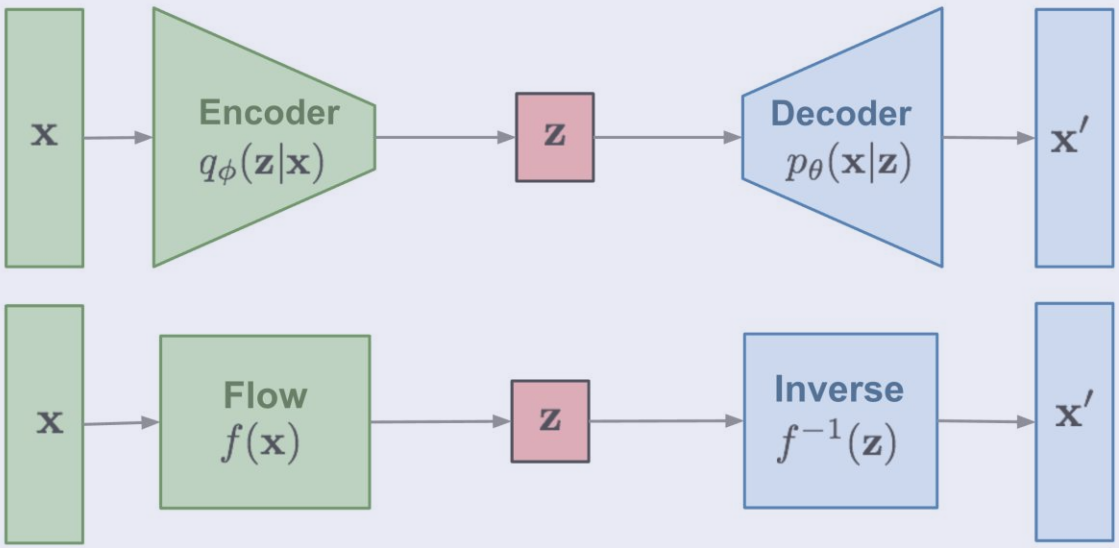
Generative Model: Normalizing Flow

Likelihood-based models

- Often require strong restrictions on model architecture (e.g., bijective function in NFs) to ensure a tractable normalizing constant for likelihood computation.
- Or must rely on surrogate objective functions to approximate maximum likelihood training

VAE: maximize variational lower bound

Flow-based models: Invertible transform of distributions



Examples:

- Bayesian Networks (e.g., VAEs), Autoregressive models, Flow models, Energy-based models (EBMs)

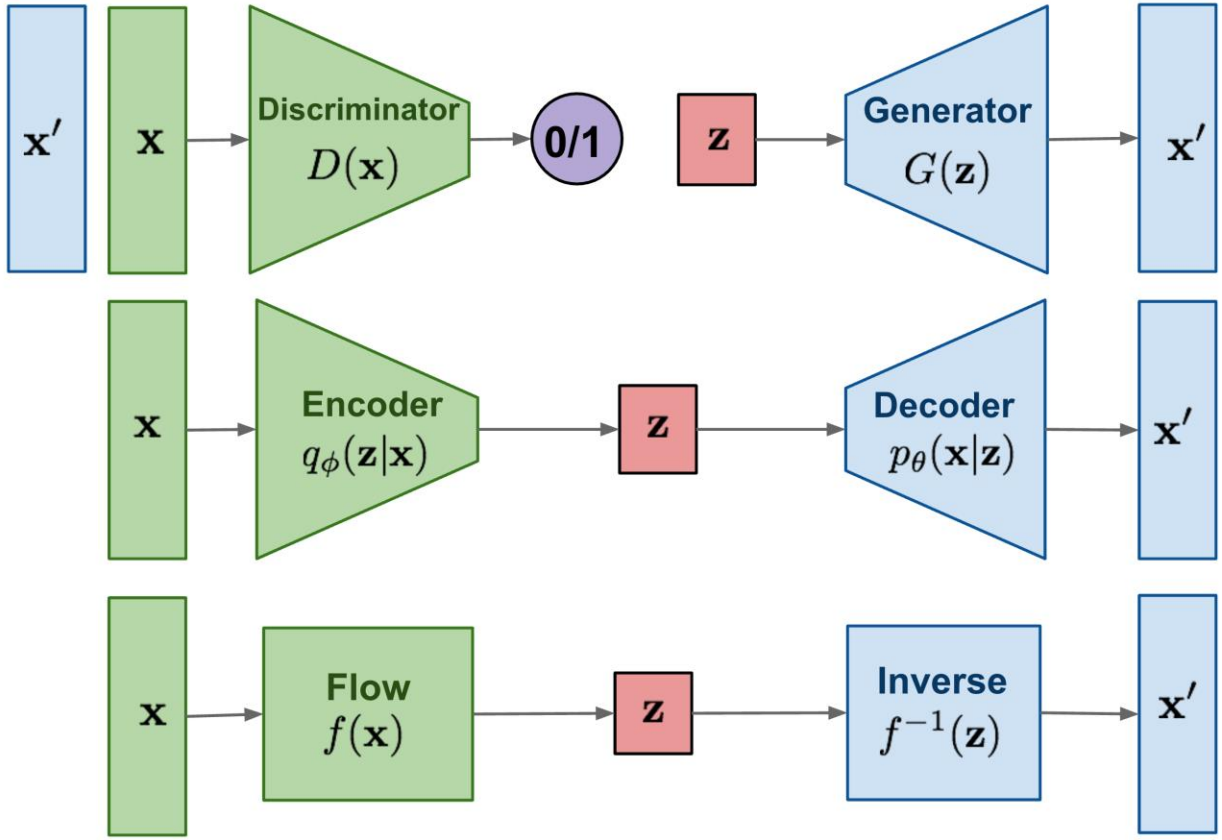
https://lilianweng.github.io/posts/2018-10-13-flow-models/

Generative Model: Normalizing Flow

GAN: Adversarial training

VAE: maximize variational lower bound

Flow-based models: Invertible transform of distributions



https://lilianweng.github.io/posts/2018-10-13-flow-models/

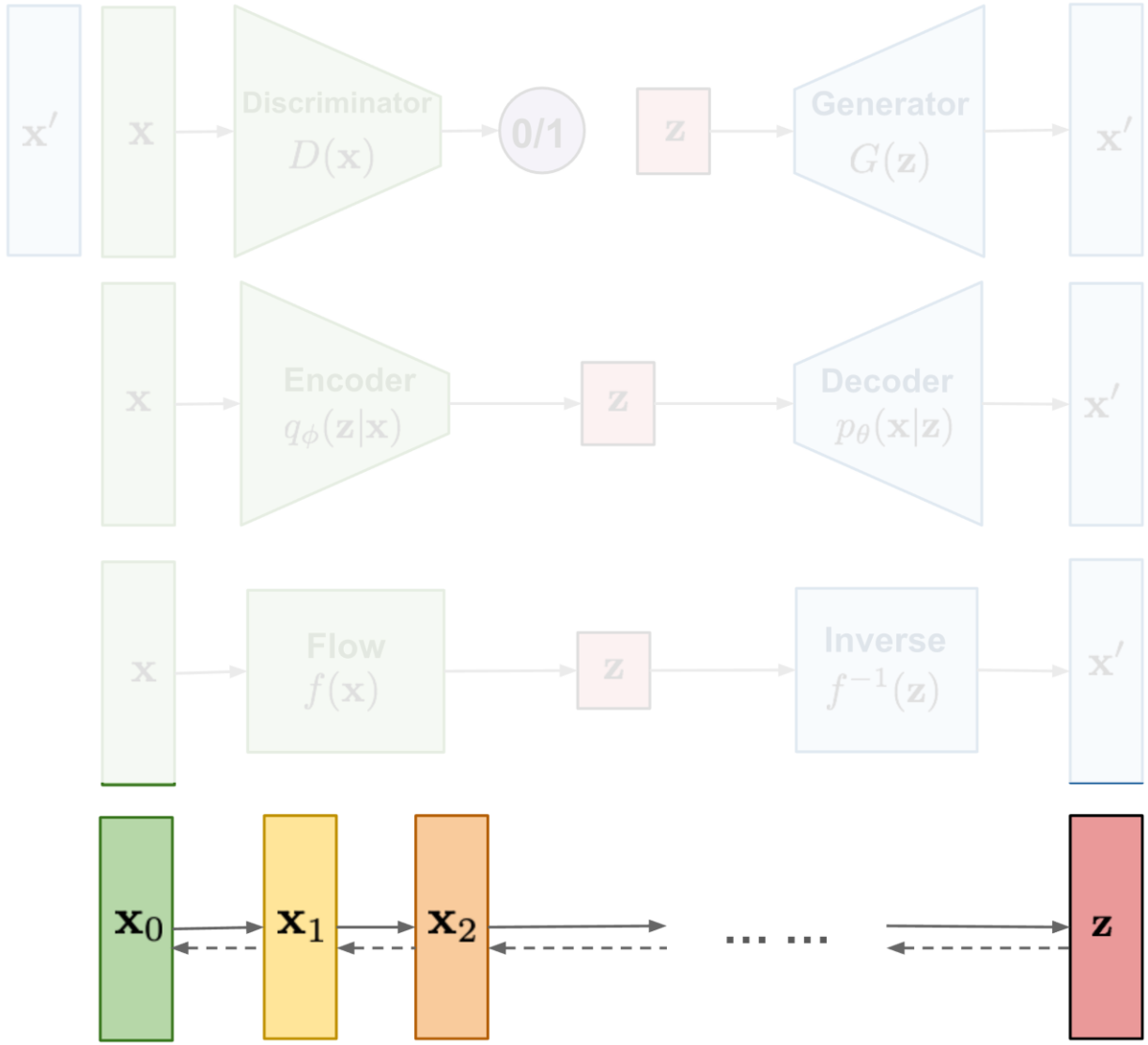
Generative Model: Diffusion Model

GAN: Adversarial training

VAE: maximize variational lower bound

Flow-based models: Invertible transform of distributions

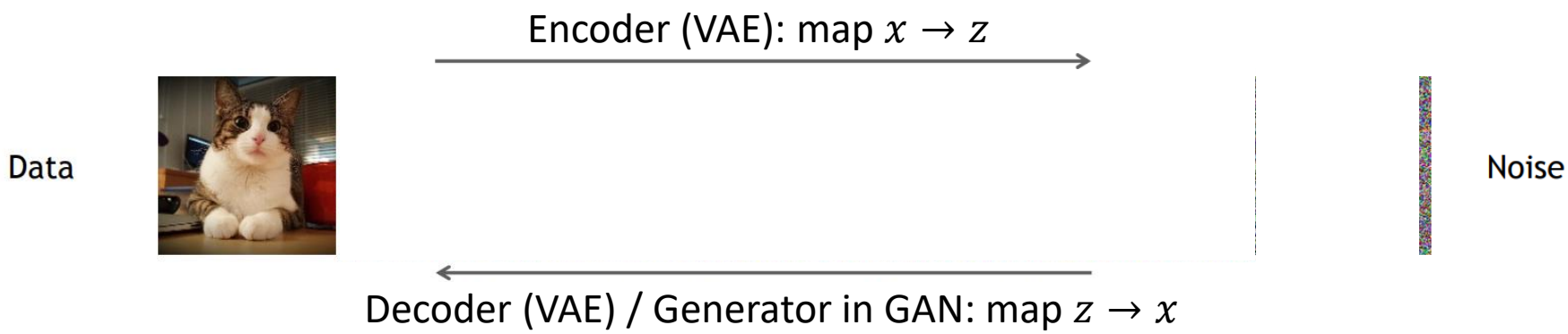
Diffusion models: Gradually add Gaussian noise and then reverse



https://lilianweng.github.io/posts/2018-10-13-flow-models/

Recap on Generative models

- Both VAEs and GANs generate new samples (e.g., new images) by following two steps:
  - Sample z from $N(0, I)$
  - Use a decoder/generator to map a sampled z to a new data x
- Two observations:
  - z is sampled from $N(0, I)$, which can be considered as a noise

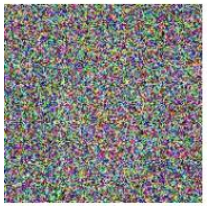


Recap on Generative models

- Both VAEs and GANs generate new samples (e.g., new images) by following two steps:
  - Sample z from $N(0, I)$
  - Use a decoder/generator to map a sampled z to a new data x
- Two observations:
  - z is sampled from $N(0, I)$, which can be considered as a noise
  - If we assume z to be the same shape as x, then essentially what you are doing is removing noise (aka denoising) z to obtain a new data x

Encoder (VAE): map $x \rightarrow z$

Data

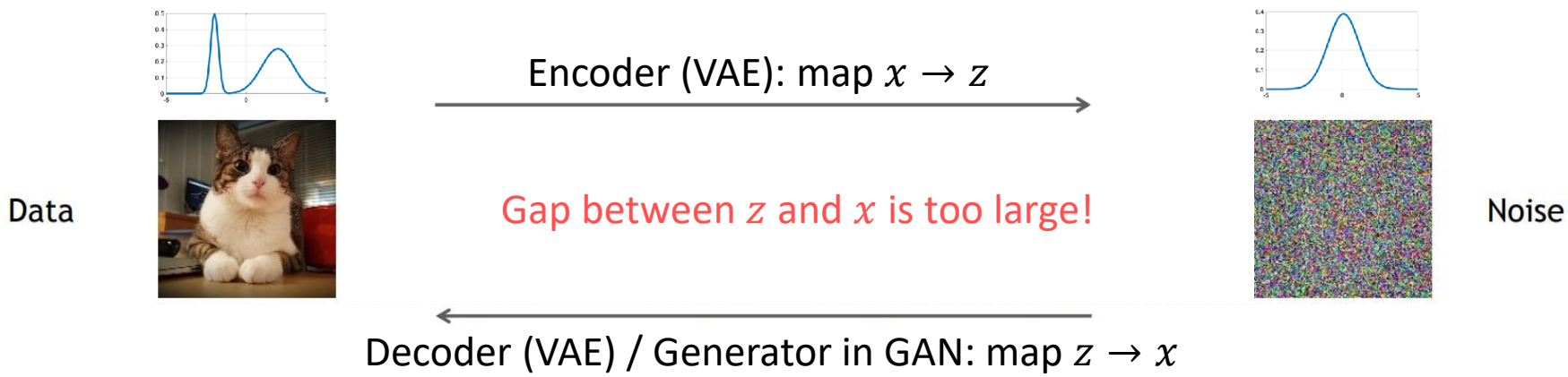


Noise

Decoder (VAE) / Generator in GAN: map $z \rightarrow x$

Recap on Generative models

- Both VAEs and GANs generate new samples (e.g., new images) by following two steps:
  - Sample z from $N(0, I)$
  - Use a decoder/generator to map a sampled z to a new data x
- Two observations:
  - z is sampled from $N(0, I)$, which can be considered as a noise
  - If we assume z to be the same shape as x, then essentially what you are doing is removing noise (aka denoising) z to obtain a new data x



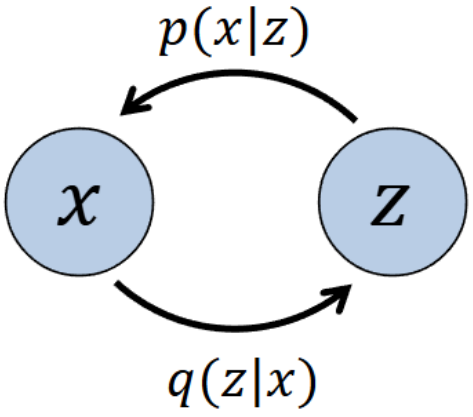
Recap on Variational Autoencoder (VAE)

- VAEs: a likelihood-based generative model

- Encoder: an inference model that approximates the posterior $q(z|x)$

- Decoder: a generative model that transforms a Gaussian variable z to real data x

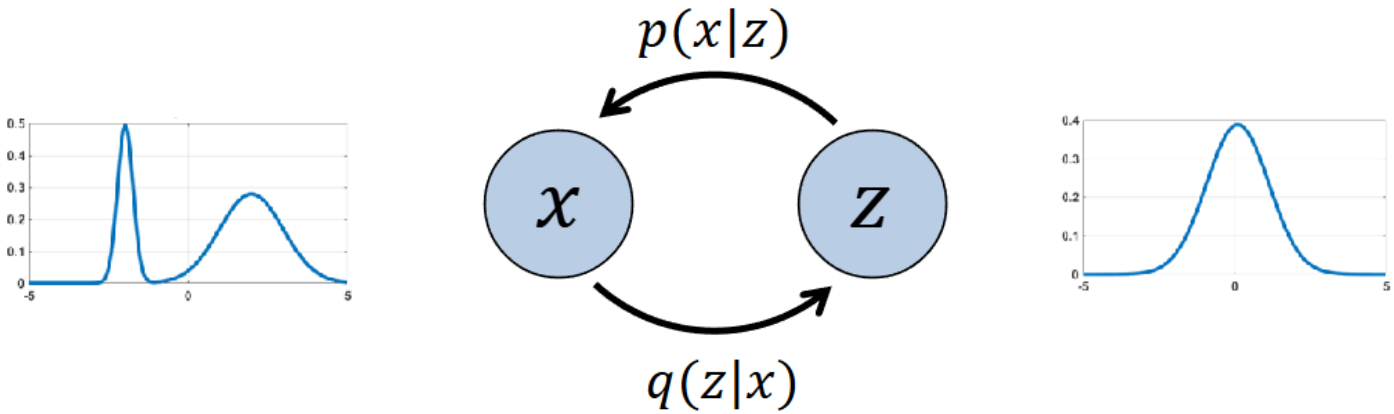
- Training: maximize the ELBO



Limitations of VAEs

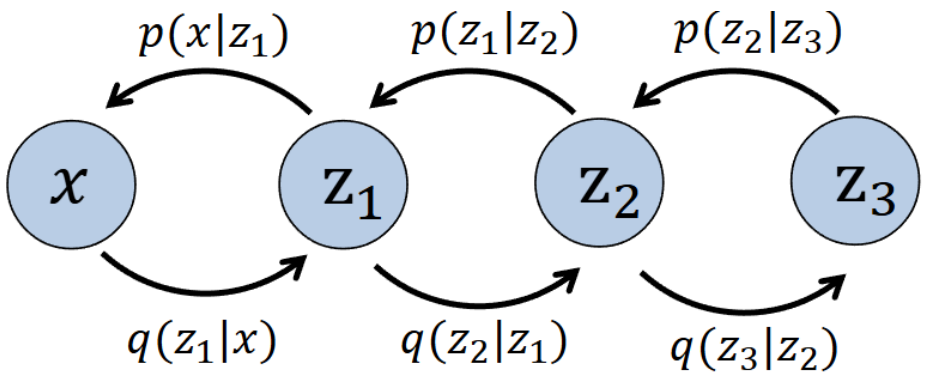
- Decoder must transform a standard Gaussian all the way to the target distribution in one-step
  - Often too large a gap

- Solution: have some intermediate latent variables to reduce the gap of each step



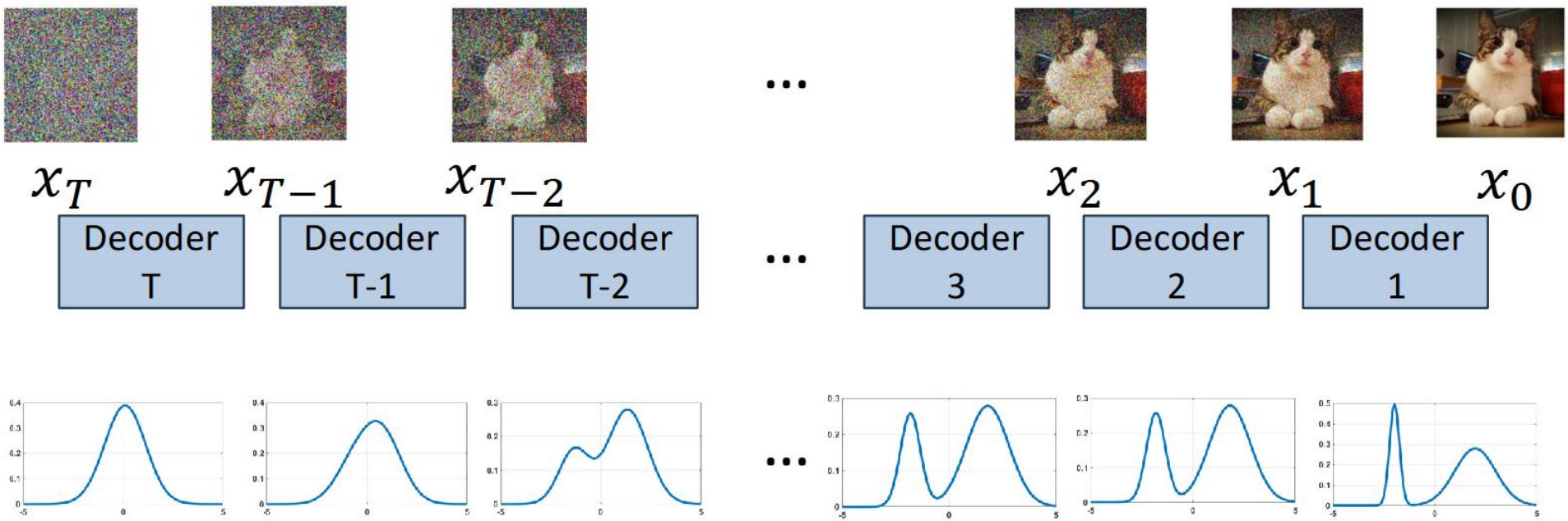
Hierarchical VAEs

- Hierarchical VAEs – Stacking VAEs on top of each other
  - Multiple (T) intermediate latent
  - Joint distribution $p(\boldsymbol{x}, \boldsymbol{z}_{1:T}) = p(\boldsymbol{z}_T) p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}_1) \prod_{t=2}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{z}_{t-1}|\boldsymbol{z}_t)$
  - Posterior $q_{\boldsymbol{\phi}}(\boldsymbol{z}_{1:T}|\boldsymbol{x}) = q_{\boldsymbol{\phi}}(\boldsymbol{z}_1|\boldsymbol{x}) \prod_{t=2}^{T} q_{\boldsymbol{\phi}}(\boldsymbol{z}_t|\boldsymbol{z}_{t-1})$
- Better likelihood achieved!



Hierarchical VAEs

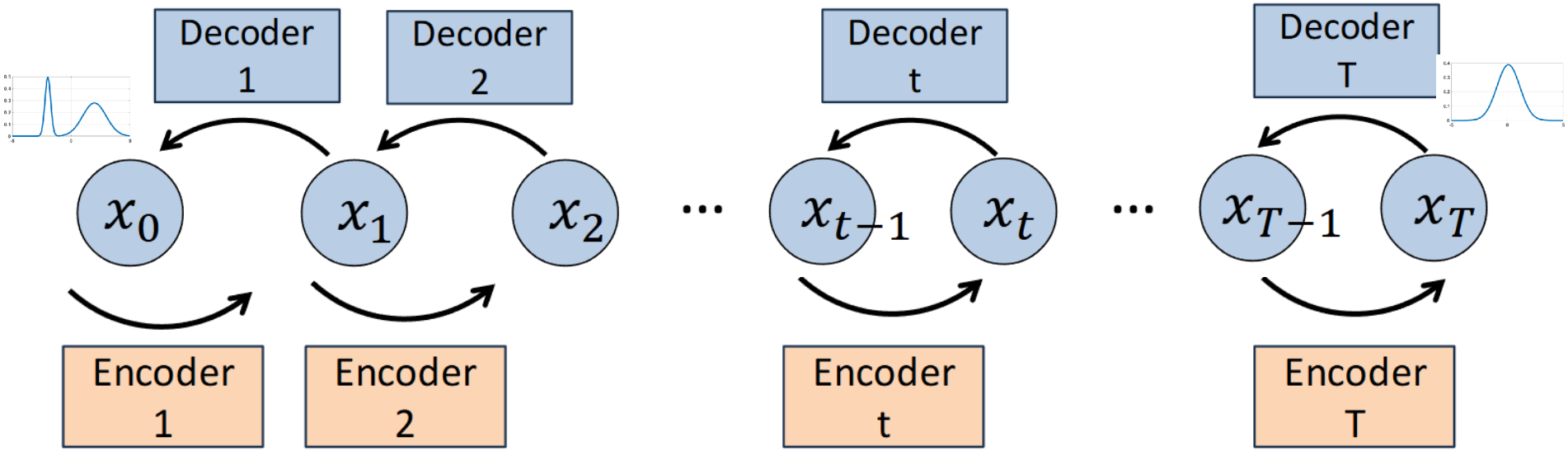
- At each step, the decoder removes part of the noise
- Provides a seed model closer to final distribution, incrementally recovering the final distribution



Hierarchical VAEs

- While each decoder incrementally “removes noise”

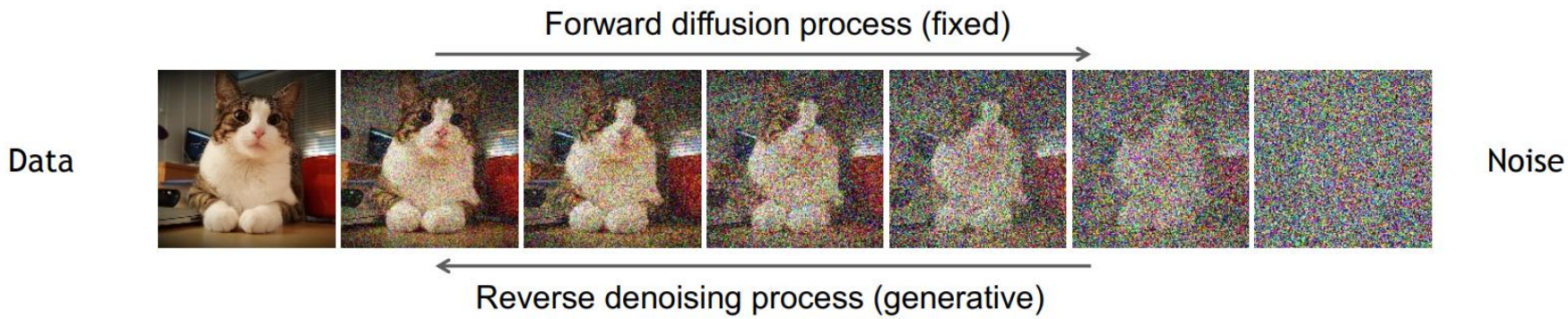
- Each encoder incrementally “adds noise”
  - Each VAE encoder is learned with KL-divergence between the posterior and the prior



Diffusion Models

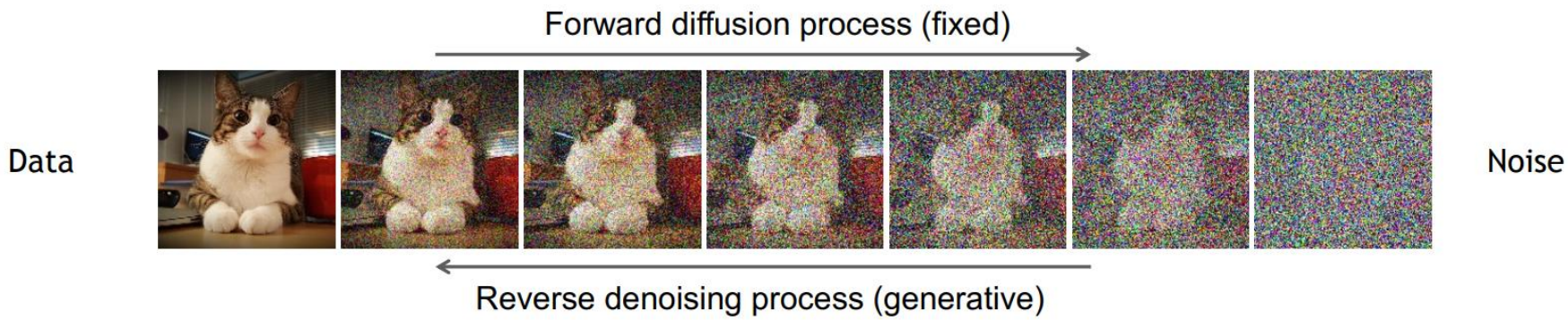
- Special case of hierarchical VAE
  - with fixed inference encoder
  - z and x have the same resolution (tensor shape)

- Core idea: Learn to generate x by incrementally learning to denoise z



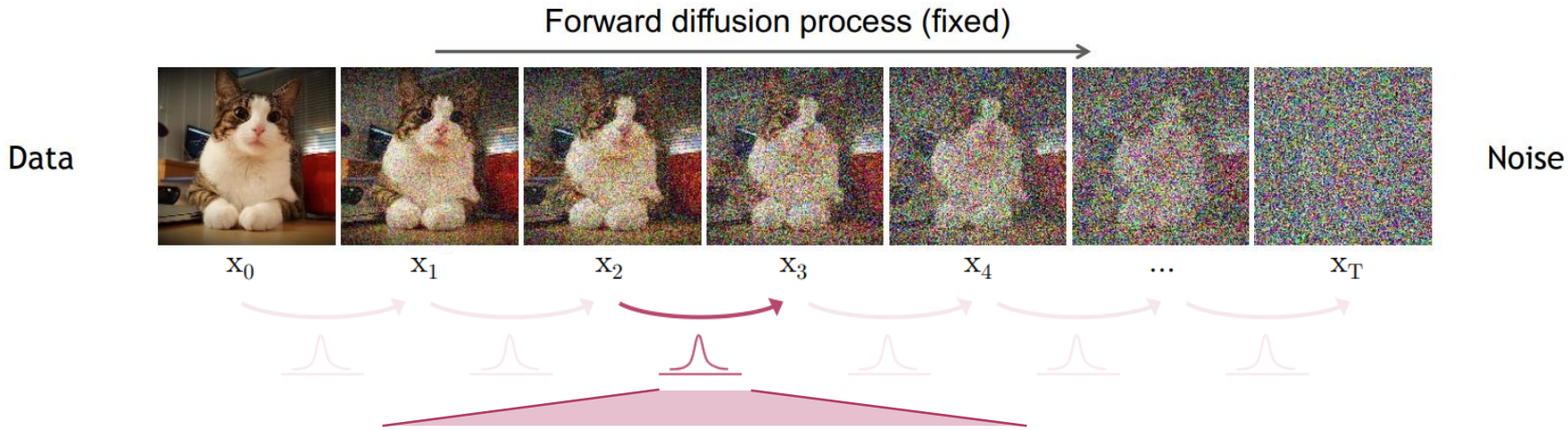
Diffusion Models

- Core idea: Learn to generate by denoising
- Denoising diffusion models consist of two processes:
  - Forward diffusion process that gradually adds noise to input
  - Reverse denoising process that learns to generate data by denoising



Diffusion Models: Forward Diffusion Process

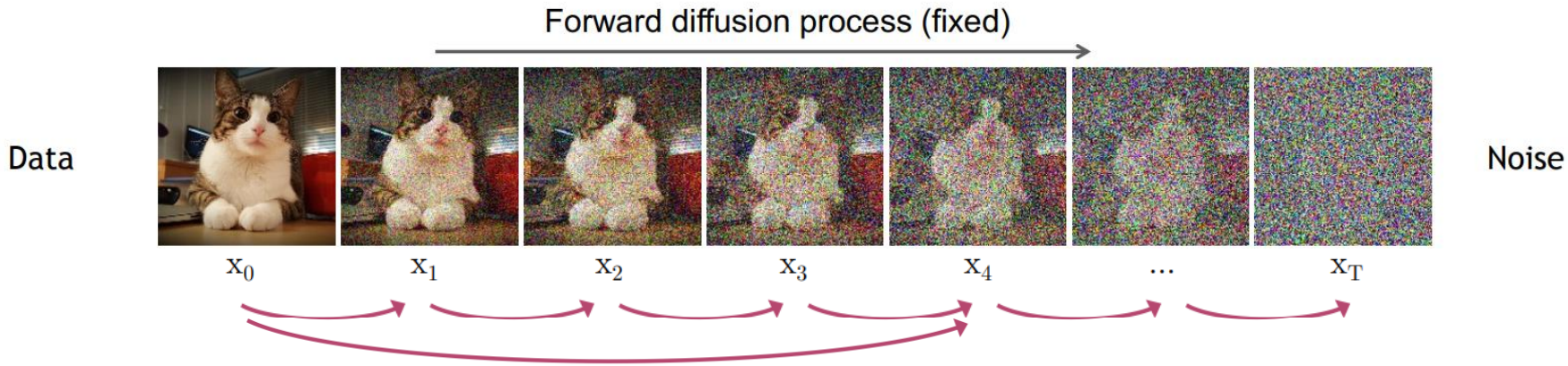
- The formal definition of the forward process in T steps:



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t\mathbf{I})$$

- Forward diffusion process is stacking **fixed** VAE encoders
  - Gradually adding Gaussian noise according to schedule β_t

Diffusion Models: Forward Diffusion Process



- The forward process allows sampling of x_t at arbitrary timestep t in closed form:

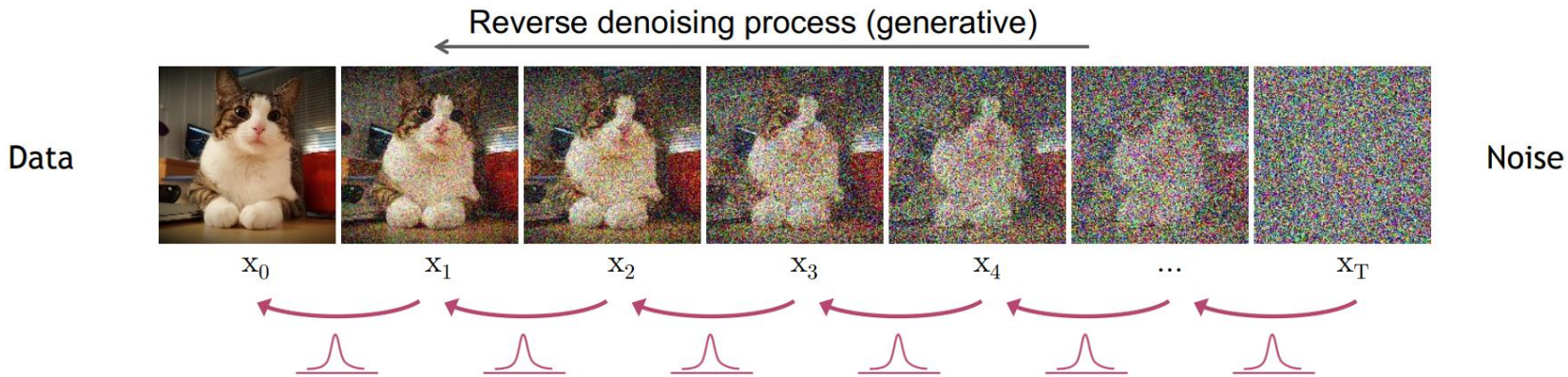
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})) \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon \quad \text{where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- The noise schedule (β_t values) is designed such that $\bar{\alpha}_T \to 0$

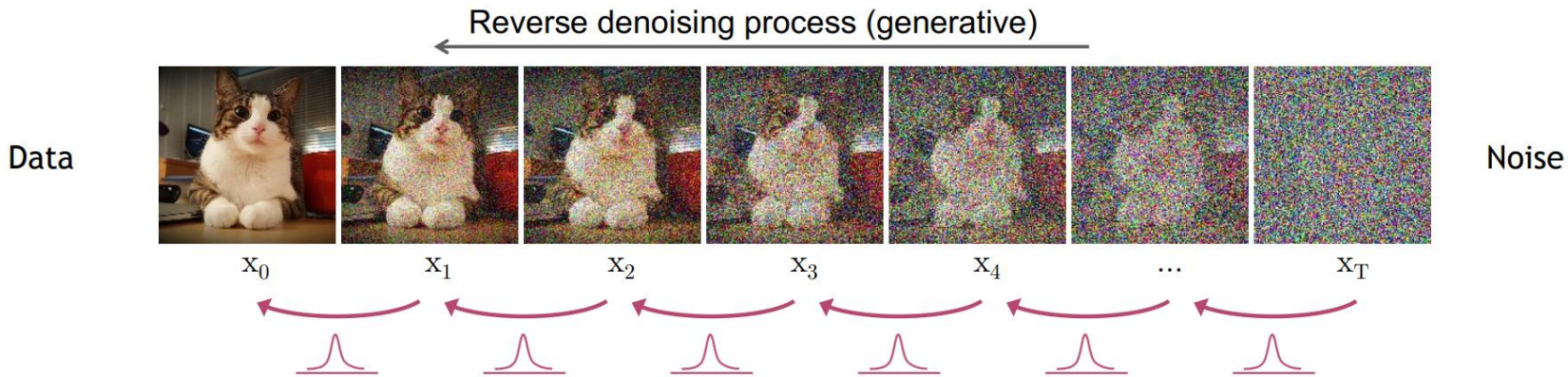
$$q(\mathbf{x}_T|\mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}))$$

Diffusion Models: Reverse Diffusion Process



- Generation process
  - Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$
  - Iteratively sample $\mathbf{x}_{t-1} \sim q(\mathbf{x}_{t-1}|\mathbf{x}_t)$
- $\mathbf{x}_{t-1} \sim q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ not directly tractable
- But it can be estimated with a Gaussian distribution if β_t is small at each step
  - The purpose of our stack of VAE decoders

Diffusion Models: Reverse Diffusion Process

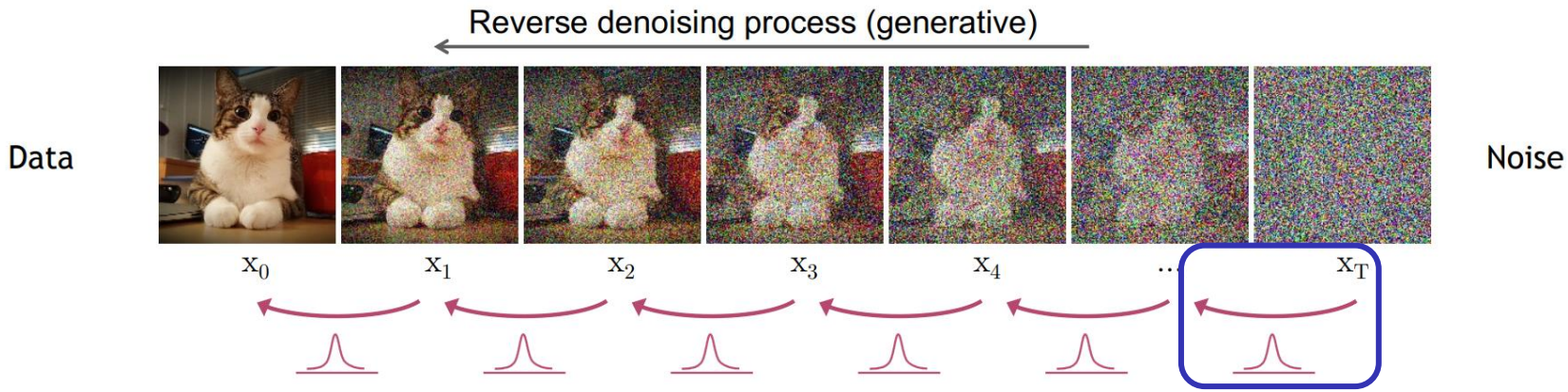


- Reverse Diffusion process is stacking learnable VAE decoders
  - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

Diffusion Models: Reverse Diffusion Process

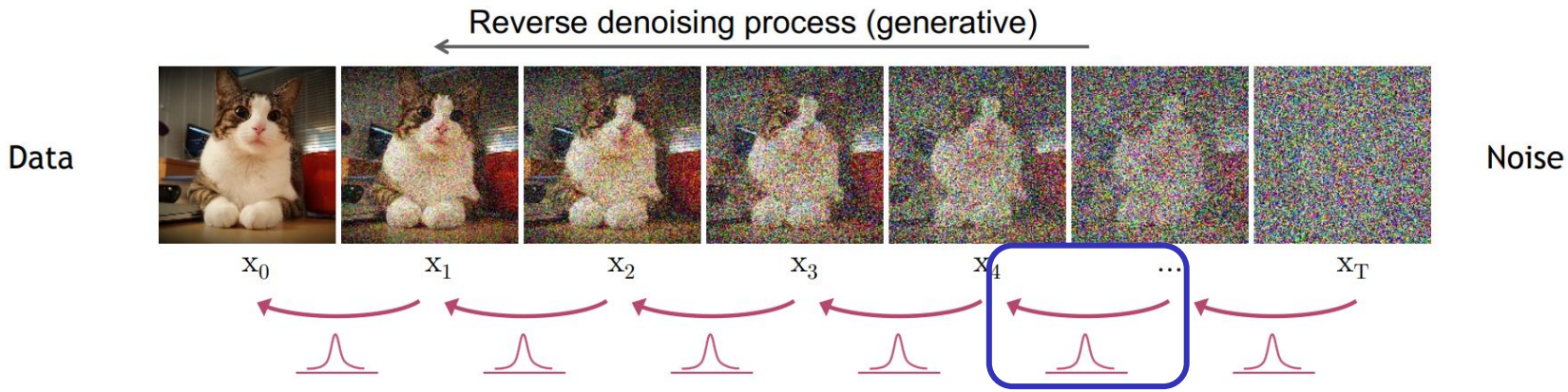


- Reverse Diffusion process is stacking learnable VAE decoders
  - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$\boxed{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})}$$

Diffusion Models: Reverse Diffusion Process

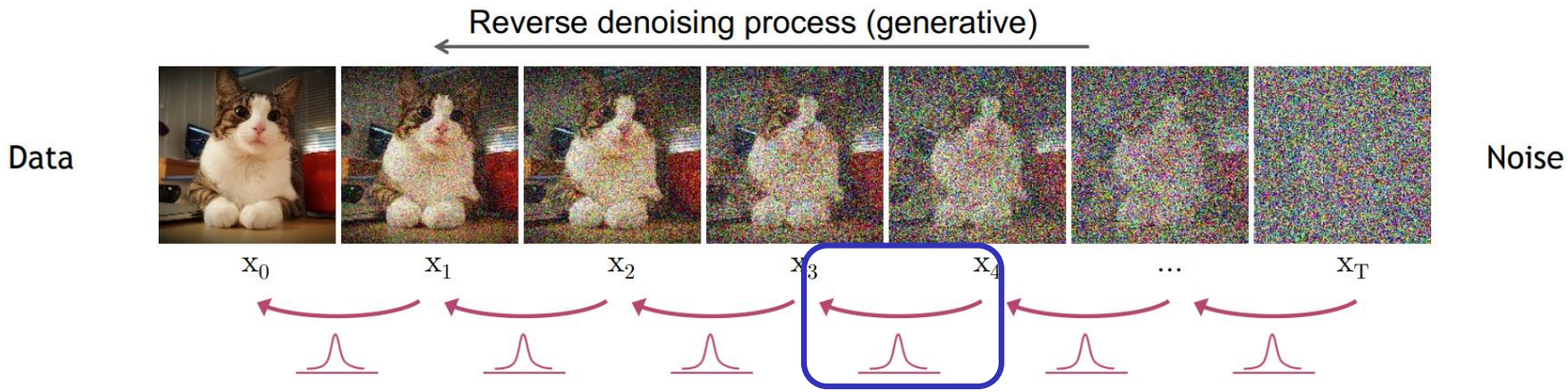


- Reverse Diffusion process is stacking learnable VAE decoders
  - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$\boxed{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})}$$

Diffusion Models: Reverse Diffusion Process

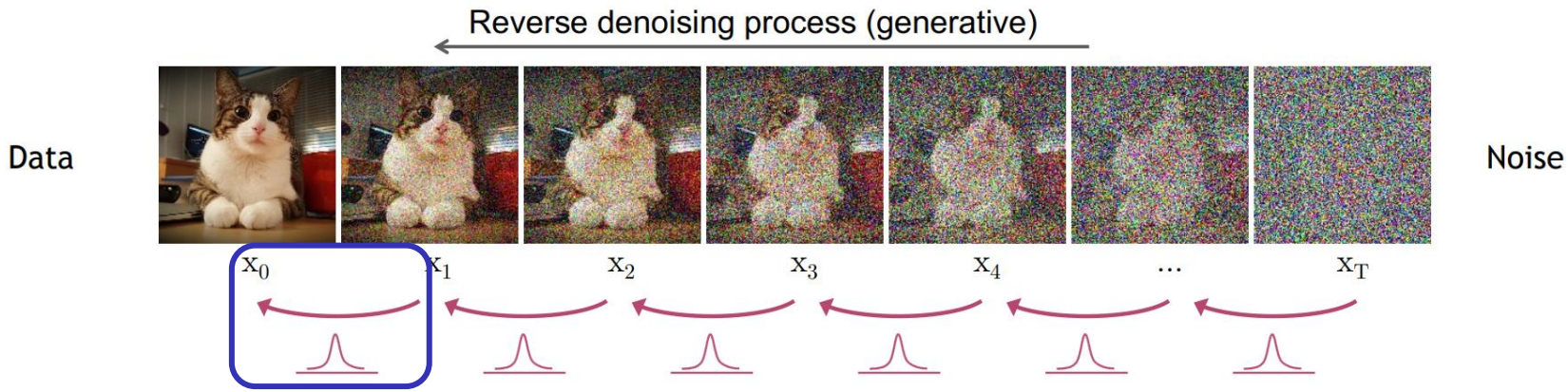


- Reverse Diffusion process is stacking learnable VAE decoders
  - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$\boxed{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})}$$

Diffusion Models: Reverse Diffusion Process

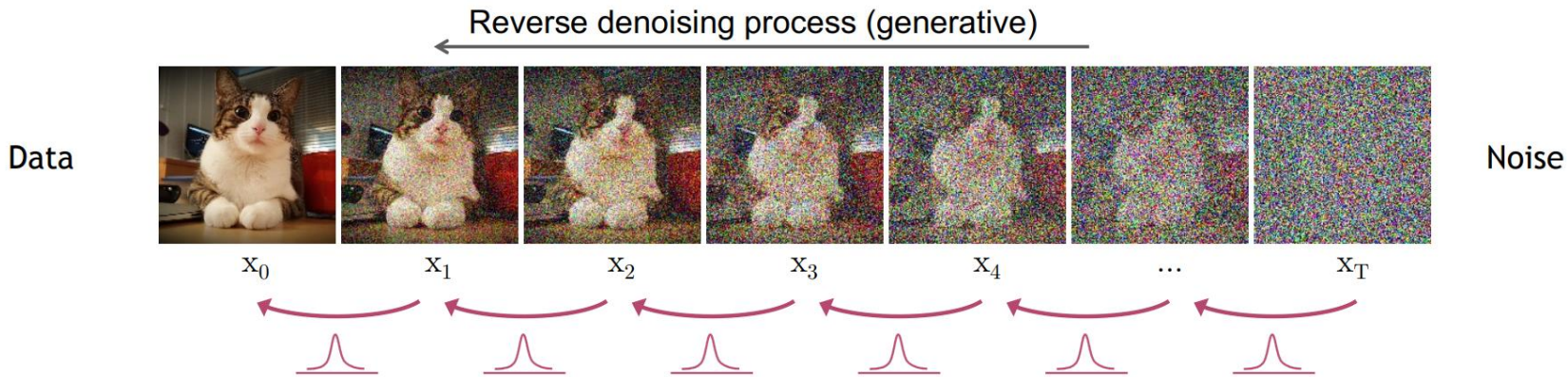


- Reverse Diffusion process is stacking learnable VAE decoders
  - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$\boxed{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})}$$

Diffusion Models: Reverse Diffusion Process



- Reverse Diffusion process is stacking learnable VAE decoders
  - Predicting the mean and std of added Gaussian Noise

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \qquad p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boxed{\mu_\theta(\mathbf{x}_t, t)}, \sigma_t^2 \mathbf{I})$$

Trainable network, shared across all timesteps

Generative Learning by Denoising

- Denoising models are trained with variational upper bound (negative ELBO), as done for training VAEs:

$$\mathbb{E}_{q(\mathbf{x}_0)}\left[-\log p_\theta(\mathbf{x}_0)\right] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] =: L$$

- Which derives to:

$$L = \mathbb{E}_q\left[\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T))}_{L_T} + \sum_{t>1}\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{-\log p_\theta(\mathbf{x}_0|\mathbf{x}_1))}_{L_0}\right]$$

constant scaling

- where $q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)$ is a tractable posterior distribution:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1};\tilde{\mu}_t(\mathbf{x}_t,\mathbf{x}_0),\tilde{\beta}_t\mathbf{I}),$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t,\mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{1-\beta_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t \text{ and } \tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$$

Parameterizing the Denoising Model

- KL Divergence between Gaussian has a simple form:

$$L_{t-1} = D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2\right] + C$$

- Recall that: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon$

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$$

- Represent the mean of the denoising model using a noise-prediction network

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1-\beta_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boxed{\epsilon_\theta(\mathbf{x}_t, t)}\right)$$

- Final objective becomes:

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0\sim q(\mathbf{x}_0), \epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon}_{\mathbf{x}_t}, t)||^2\right] + C$$

Simplified Training Objective

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\underbrace{\frac{\beta_t^2}{2\sigma_t^2 (1-\beta_t)(1-\bar{\alpha}_t)}}_{\lambda_t} ||\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, t)||^2 \right] + C$$

- λ_t ensures that the training objective is weighted properly for the maximum data likelihood training.
  - However, this weight is often very large for small t's (again, another gap between theory and practice).

- This is further simplified by setting λ_t to 1 (improving sample quality):

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1,T)} \left[||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon}_{\mathbf{x}_t}, t)||^2 \right]$$

Summary: Training and Sampling

Algorithm 1 Training

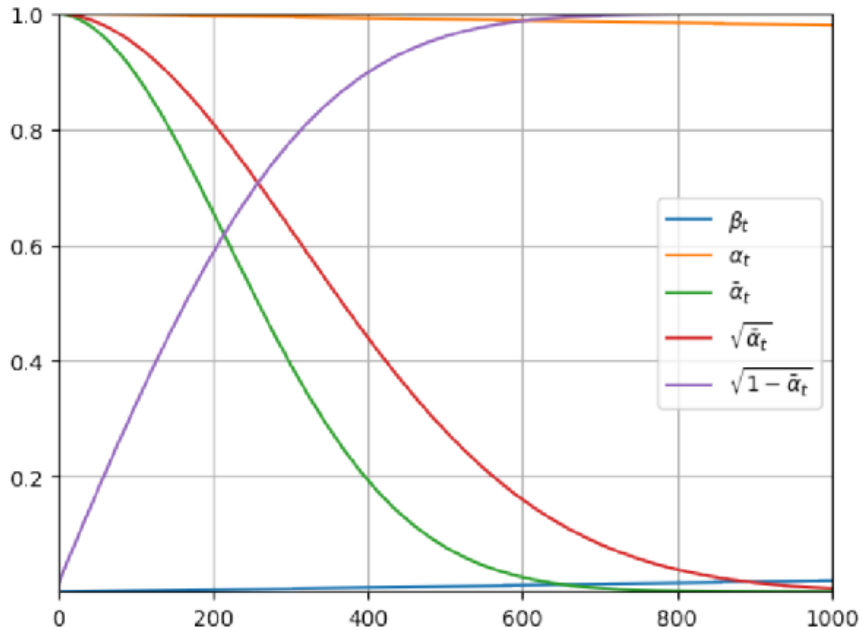
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

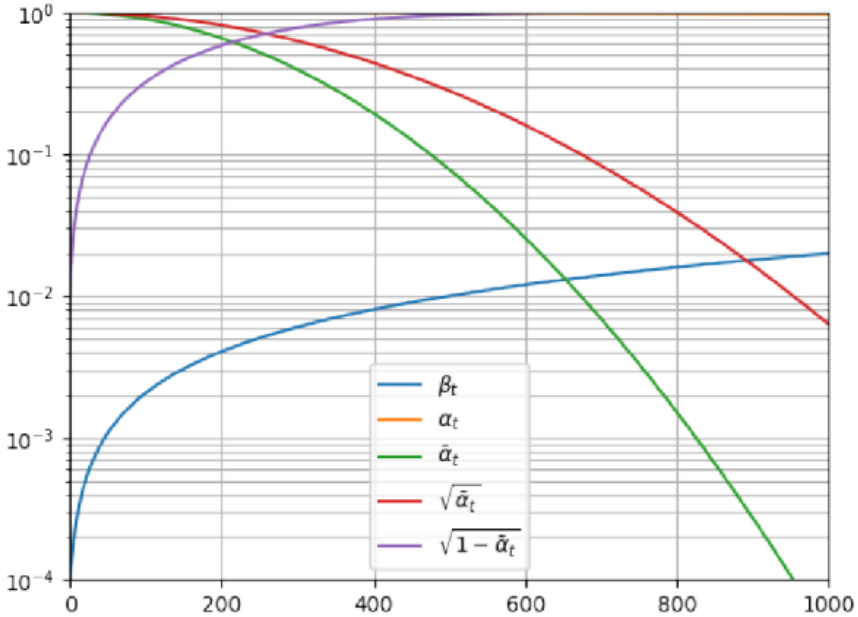
Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Summary: Noise Schedule

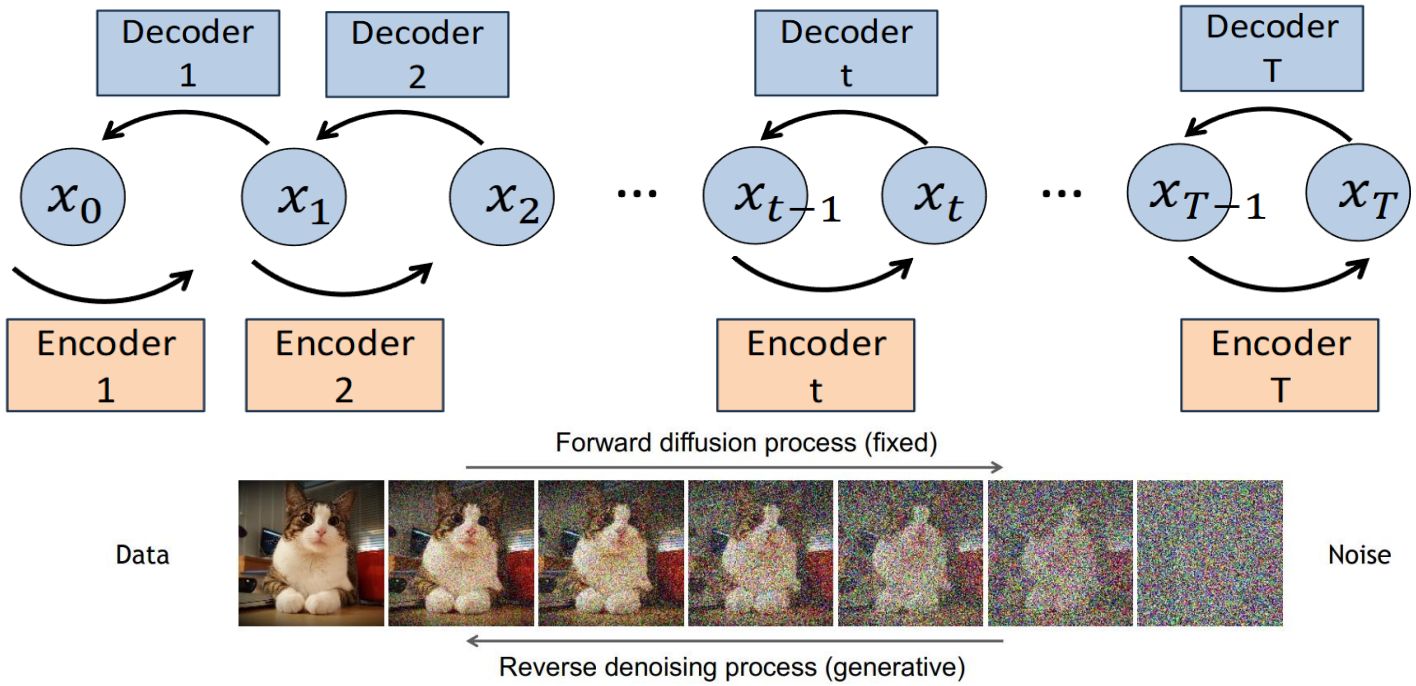
- The right plot is the same as left plot but with log-scale on y-axis.





Connection with Hierarchical VAEs

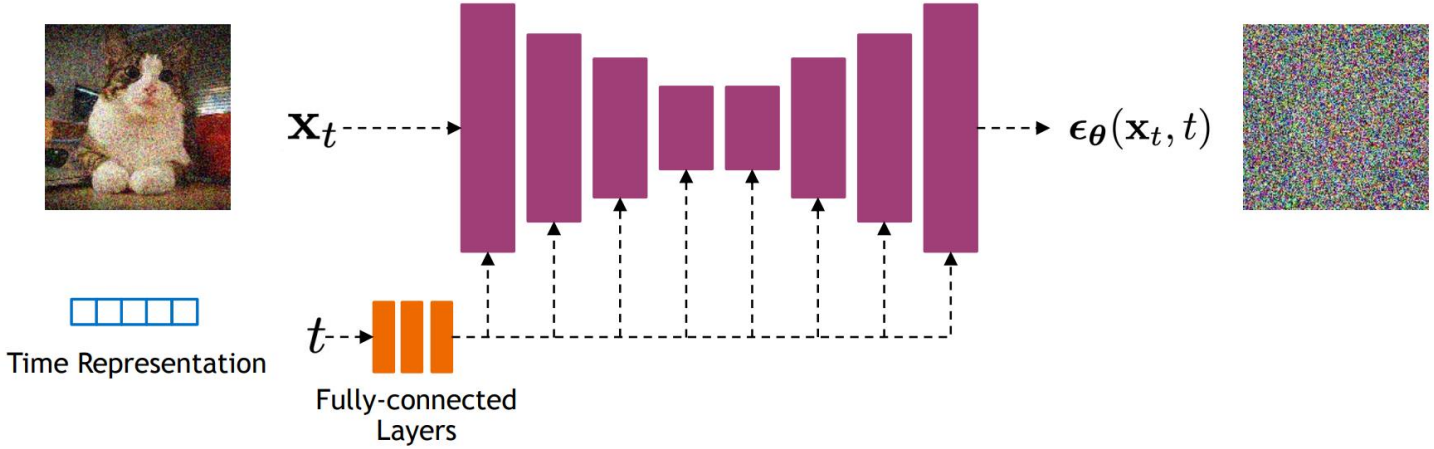
- Diffusion model is a special case of Hierarchical VAEs
  - Fixed inference models in forward process
  - Latent variables have the same dimension as data
  - ELBO is decomposed to each timestep: faster to train
  - Model is trained with some weighting of ELBO



Implementation Considerations

Network Architectures

- Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers for noise prediction network



- Time representation: sinusoidal positional embeddings or random Fourier features

- Time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers

Diffusion Models from Stochastic Differential Equations and Score Matching Perspective

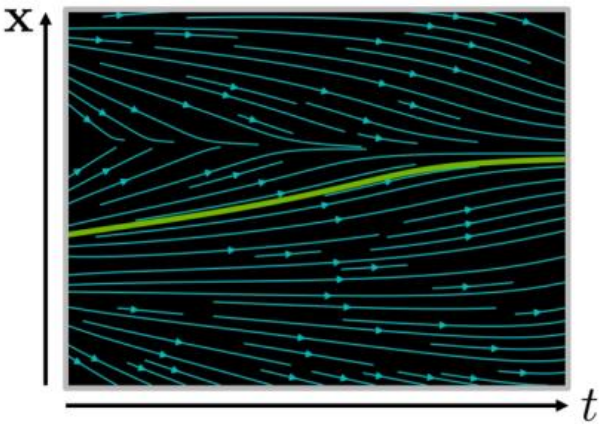
Why Stochastic Differential Equations (SDEs)?

- A unified framework for interpreting diffusion models and score-based generation models
  - Variants of diffusion-based and flow-based models

Stochastic Differential Equations

Ordinary Differential Equation (ODE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}, t) \quad \text{or} \quad \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t$$



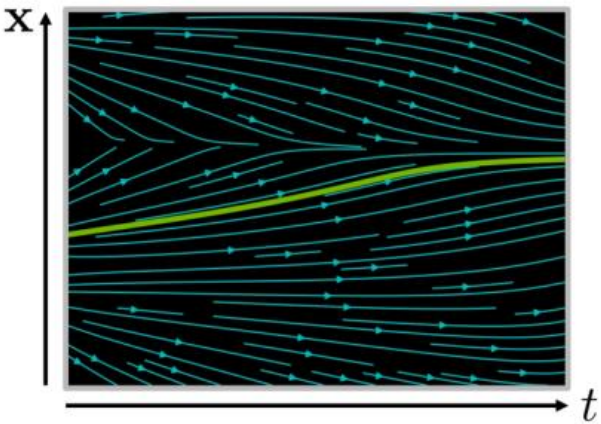
Analytical Solution: $$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau)\mathrm{d}\tau$$

Iterative Numerical $$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t$$

Stochastic Differential Equations

Ordinary Differential Equation (ODE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}, t) \quad \text{or} \quad \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t$$



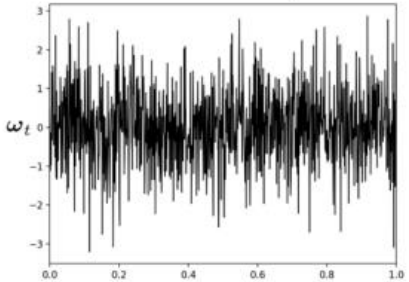
Analytical Solution: $\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau)\mathrm{d}\tau$

Iterative Numerical $\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t$

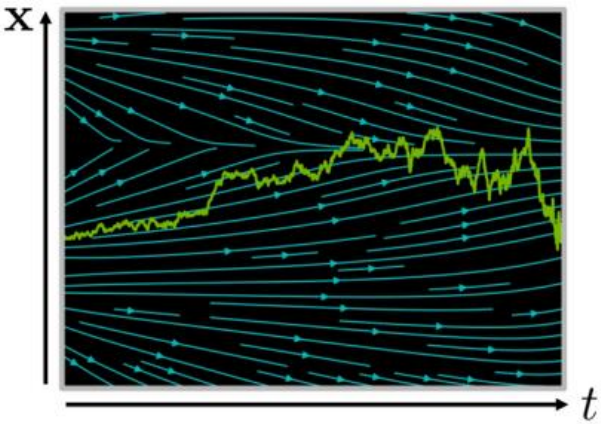
Stochastic Differential Equation (SDE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \underbrace{\mathbf{f}(\mathbf{x}, t)}_{\text{drift coefficient}} + \underbrace{\sigma(\mathbf{x}, t)}_{\text{diffusion coefficient}}\boldsymbol{\omega}_t$$

Wiener Process (Gaussian White Noise)



$$\left(\mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + \sigma(\mathbf{x}, t)\mathrm{d}\boldsymbol{\omega}_t \right)$$



$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t + \sigma(\mathbf{x}(t), t)\sqrt{\Delta t}\,\mathcal{N}(\mathbf{0}, \mathbf{I})$$

Score Matching

- General form of probability density function

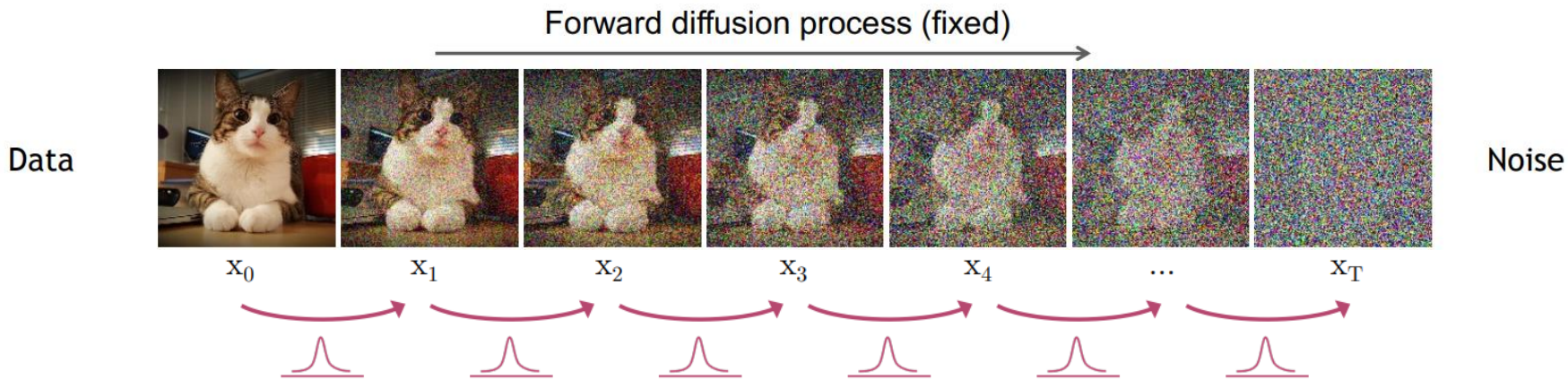
$$p_\theta(\mathbf{x}) = \frac{e^{-f_\theta(\mathbf{x})}}{Z_\theta}$$

- Maximizing the log-likelihood requires us to know
  - Often intractable

- Instead, we can model the score function

$$\nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Forward Diffusion Processes as SDEs

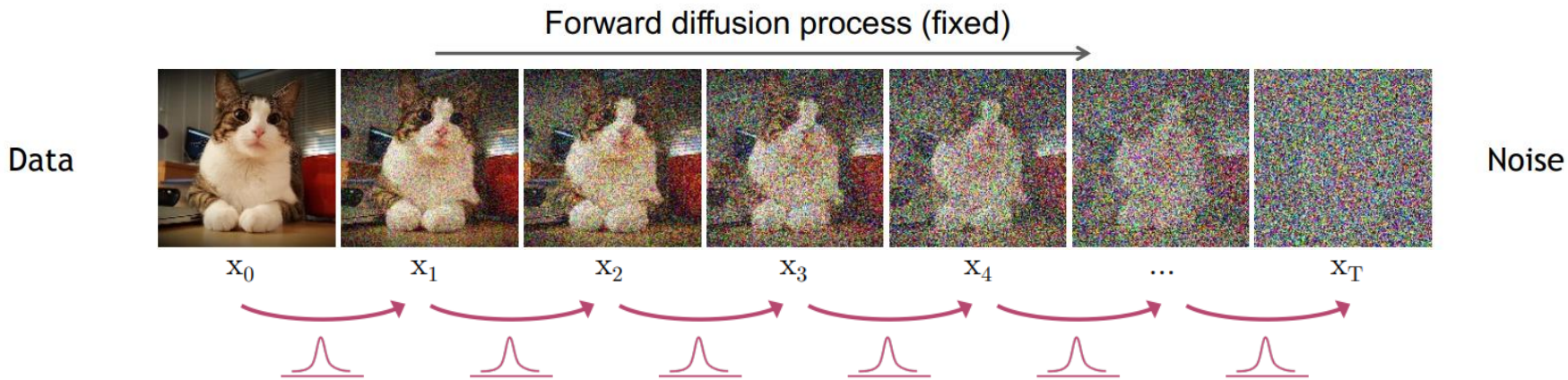


- Consider a forward process with many *many* small steps (continuous time)

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t \mathbf{I}\right)$$

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$

Forward Diffusion Processes as SDEs



- Consider a forward process with many *many* small steps (continuous time)

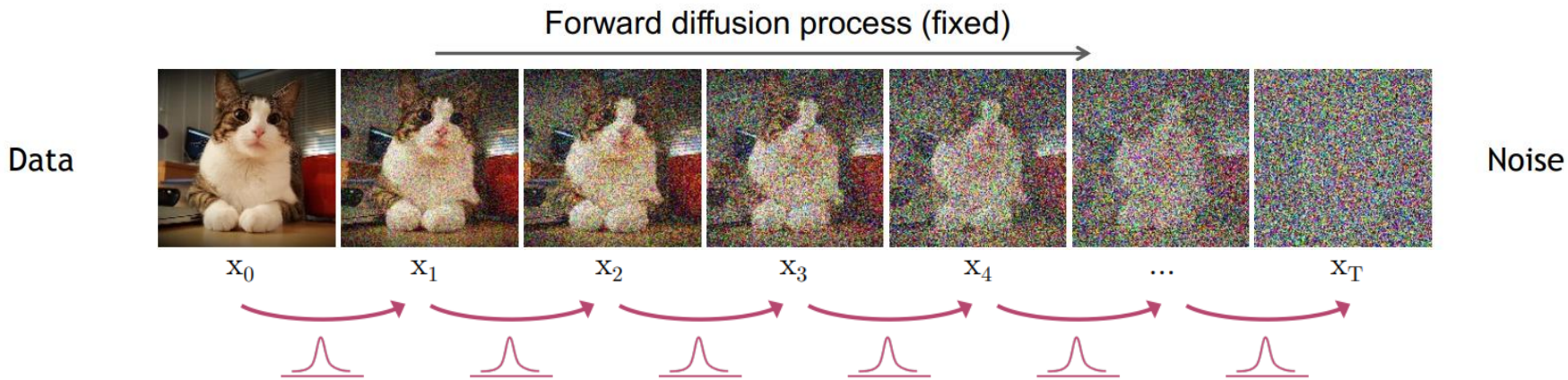
$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t \mathbf{I}\right)$$

$$\begin{aligned}\mathbf{x}_t &= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(\mathbf{0},\mathbf{I}) \\ &= \sqrt{1-\beta(t)\Delta t}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\mathcal{N}(\mathbf{0},\mathbf{I})\end{aligned} \qquad (\beta_t := \beta(t)\Delta t)$$

Allows different size along t

Step size

Forward Diffusion Processes as SDEs

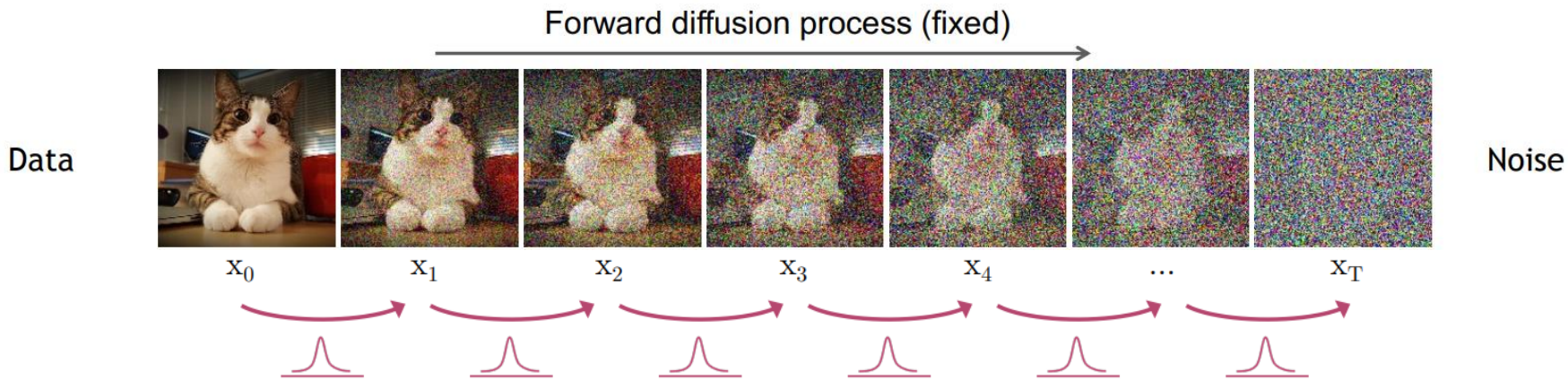


- Consider a forward process with many *many* small steps (continuous time)

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t \mathbf{I}\right)$$

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(\mathbf{0},\mathbf{I}) \\
&= \sqrt{1-\beta(t)\Delta t}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\mathcal{N}(\mathbf{0},\mathbf{I}) \qquad (\beta_t := \beta(t)\Delta t) \\
&\approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\mathcal{N}(\mathbf{0},\mathbf{I}) \qquad \text{Taylor expansion}
\end{aligned}$$

Forward Diffusion Processes as SDEs



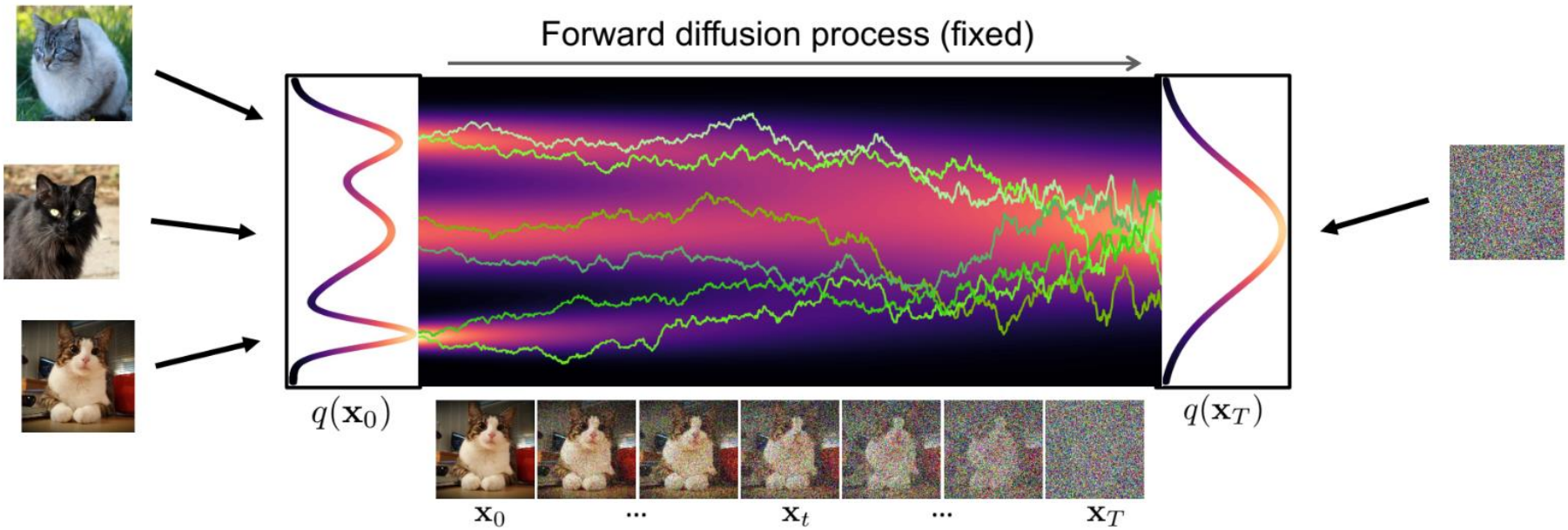
- An iterative update that can be viewed as SDEs

$$\mathbf{x}_t \approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\mathrm{d}\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t + \sqrt{\beta(t)}\mathrm{d}\boldsymbol{\omega}_t$$

Stochastic Differential Equation (SDE)

Forward Diffusion Processes as SDEs

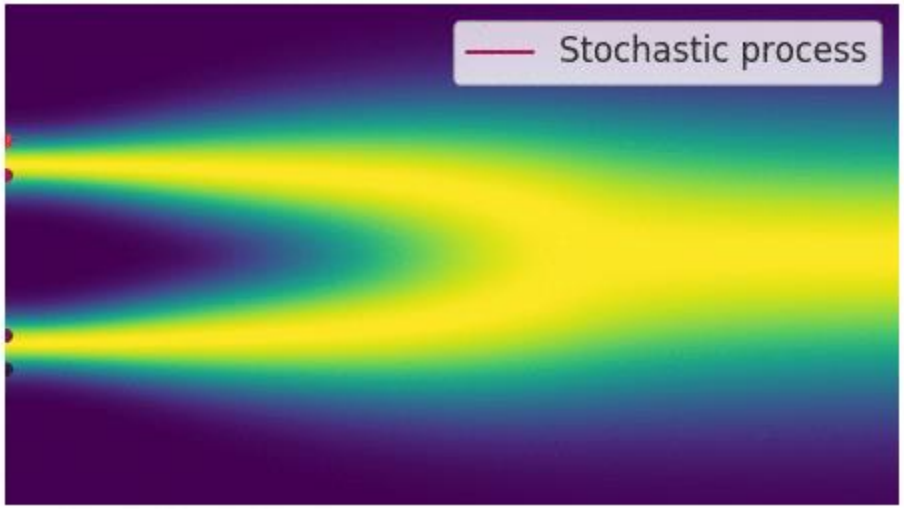


$$\mathrm{d}\mathbf{x}_t = \underbrace{-\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t}_{\text{Drift Term}} + \underbrace{\sqrt{\beta(t)}\mathrm{d}\boldsymbol{\omega}_t}_{\text{Diffusion Term}}$$

Drift Term (Pulls toward the mode)

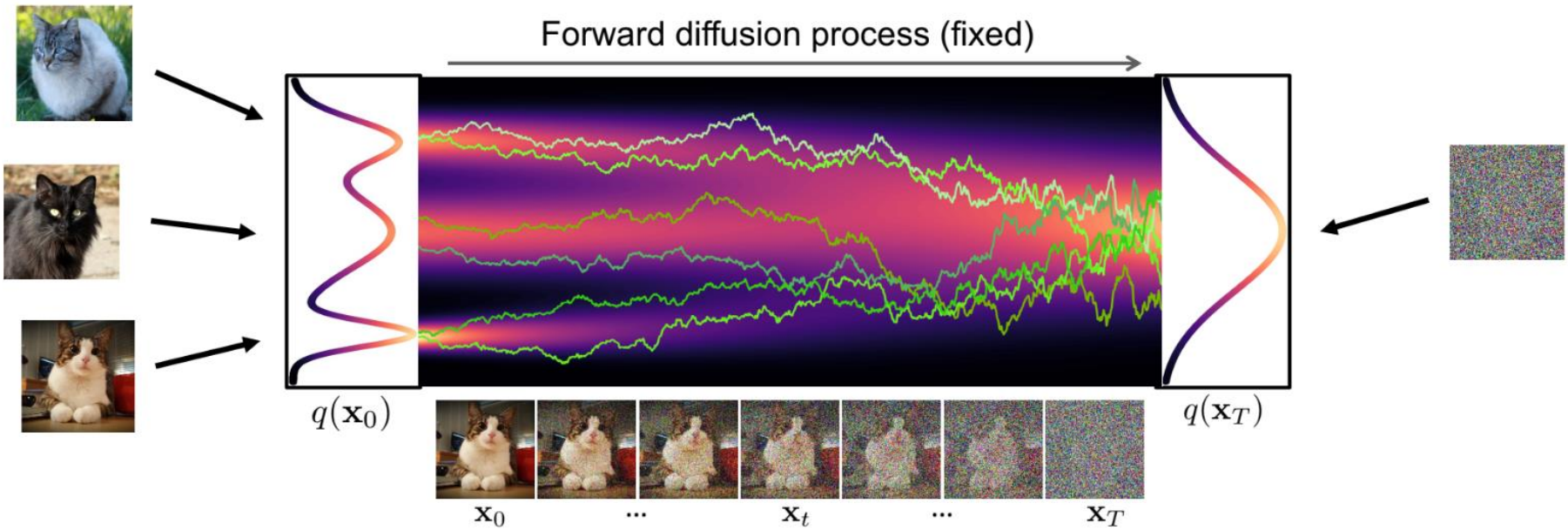
Diffusion Term (Injects Noise)

Forward Diffusion Processes as SDEs



https://yang-song.net/blog/2021/score/

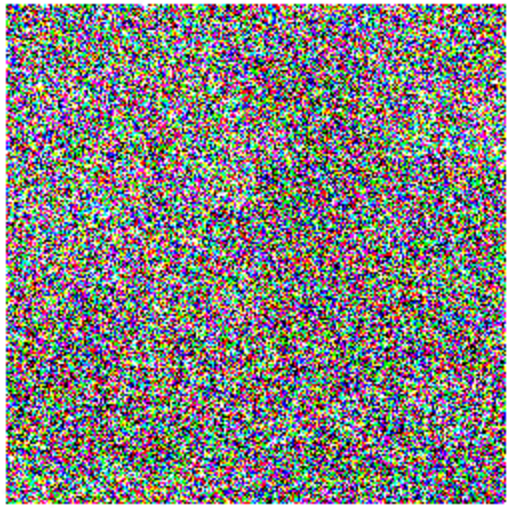
Generative Reverse SDEs

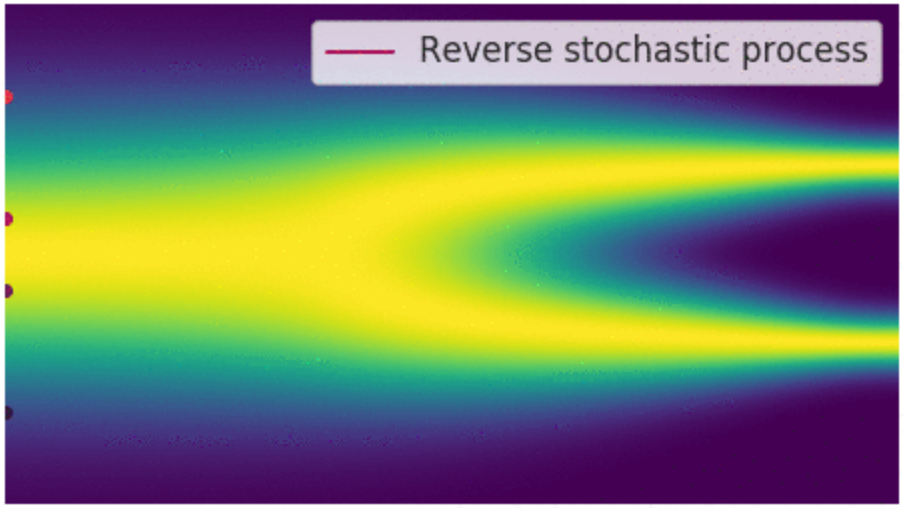


- The forward SDE has a reverse form:

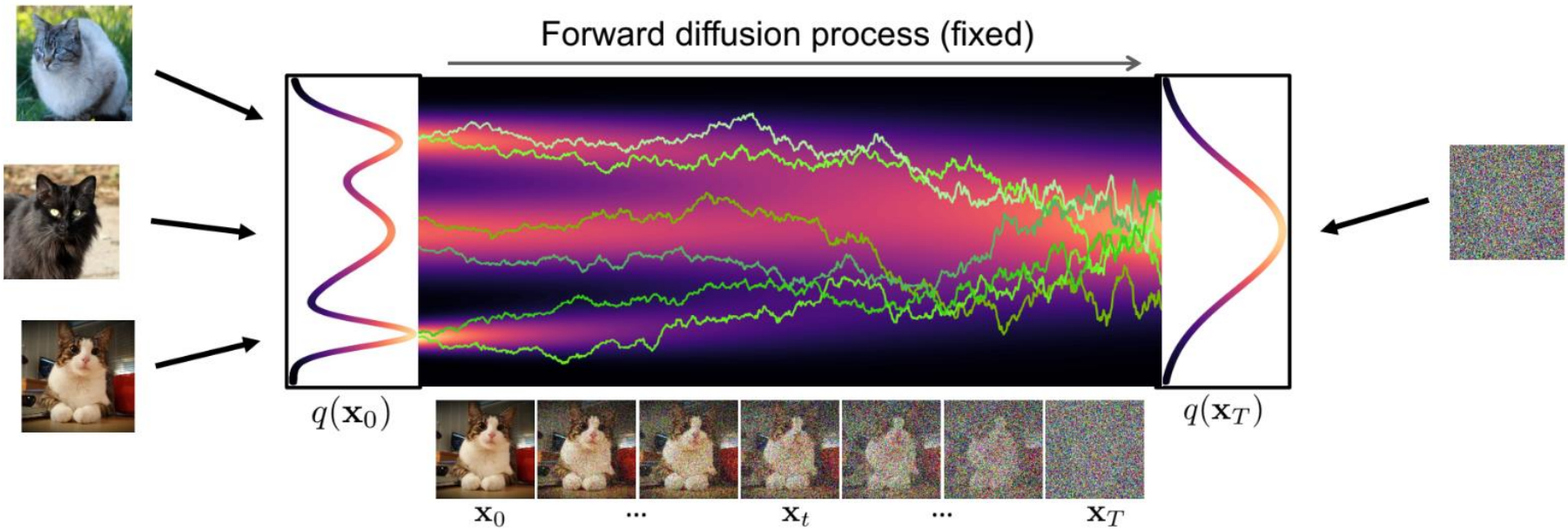
$$\mathrm{d}\mathbf{x}_t = \left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t)\nabla_{\mathbf{x}_t}\log q_t\left(\mathbf{x}_t\right)\right]\mathrm{d}t + \sqrt{\beta(t)}\mathrm{d}\overline{\boldsymbol{\omega}}_t$$

Generative Reverse SDEs





Generative Reverse SDEs



- The forward SDE has a reverse form:

Score function
How to get it?

$$\mathrm{d}\mathbf{x}_t = \left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t)\boxed{\nabla_{\mathbf{x}_t}\log q_t\left(\mathbf{x}_t\right)}\right]\mathrm{d}t + \sqrt{\beta(t)}\mathrm{d}\overline{\boldsymbol{\omega}}_t$$

Denoising Score Matching

Looks familiar?

- Denoising score matching objective

$$\min_{\boldsymbol{\theta}} \underbrace{\mathbb{E}_{t\sim\mathcal{U}(0,T)}}_{\substack{\text{diffusion}\\ \text{time } t}} \underbrace{\mathbb{E}_{\mathbf{x}_0\sim q_0(\mathbf{x}_0)}}_{\substack{\text{data}\\ \text{sample } \mathbf{x}_0}} \underbrace{\mathbb{E}_{\mathbf{x}_t\sim q_t(\mathbf{x}_t|\mathbf{x}_0)}}_{\substack{\text{diffused data}\\ \text{sample } \mathbf{x}_t}} \underbrace{\tilde{w}(t)}_{\substack{\text{weighting}\\ \text{function}}} \cdot \cdot \underbrace{\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)}_{\substack{\text{neural}\\ \text{network}}} - \underbrace{\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t \mid \mathbf{x}_0)}_{\substack{\text{score of diffused}\\ \text{data sample}}}\|_2^2$$

- Re-parametrized sampling:

$$\mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- Score function:

$$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t \mid \mathbf{x}_0) = -\nabla_{\mathbf{x}_t} \frac{(\mathbf{x}_t - \alpha_t \mathbf{x}_0)^2}{2\sigma_t^2} = -\frac{\mathbf{x}_t - \alpha_t \mathbf{x}_0}{\sigma_t^2} = -\frac{\alpha_t \mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon} - \alpha_t \mathbf{x}_0}{\sigma_t^2} = -\frac{\boldsymbol{\epsilon}}{\sigma_t}$$

- Denoising network:

$$\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) := -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)}{\sigma_t}$$

- Final objective:

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{t\sim\mathcal{U}(0,T)} \mathbb{E}_{\mathbf{x}_0\sim q_0(\mathbf{x}_0)} \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})} \hat{w}(t) \cdot \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|_2^2 \quad \hat{w}(t) = \frac{\tilde{w}(t)}{\sigma_t}$$

Weighted Diffusion Objective

- Denoising score matching objective with loss weighting

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{t\sim\mathcal{U}(0,T)}\mathbb{E}_{\mathbf{x}_0\sim q_0(\mathbf{x}_0)}\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\frac{\lambda(t)}{\sigma_t^2}\left\|\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\mathbf{x}_t,t\right)\right\|_2^2$$

- Re-parametrized sampling:

$$\mathbf{x}_t = \alpha_t\mathbf{x}_0 + \sigma_t\boldsymbol{\epsilon} \quad \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})$$

- Score function:

$$\nabla_{\mathbf{x}_t}\log q_t\left(\mathbf{x}_t \mid \mathbf{x}_0\right) = -\nabla_{\mathbf{x}_t}\frac{\left(\mathbf{x}_t-\alpha_t\mathbf{x}_0\right)^2}{2\sigma_t^2} = -\frac{\mathbf{x}_t-\alpha_t\mathbf{x}_0}{\sigma_t^2} = -\frac{\alpha_t\mathbf{x}_0+\sigma_t\boldsymbol{\epsilon}-\alpha_t\mathbf{x}_0}{\sigma_t^2} = -\frac{\boldsymbol{\epsilon}}{\sigma_t}$$

- Denoising network:

$$\mathbf{s}_{\boldsymbol{\theta}}\left(\mathbf{x}_t,t\right) := -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\mathbf{x}_t,t\right)}{\sigma_t}$$

- Final objective:

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{t\sim\mathcal{U}(0,T)}\mathbb{E}_{\mathbf{x}_0\sim q_0(\mathbf{x}_0)}\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\hat{w}(t)\cdot\left\|\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\mathbf{x}_t,t\right)\right\|_2^2 \quad \hat{w}(t) = \frac{\tilde{w}(t)}{\sigma_t}$$

Conditional Diffusion Models

- Unconditional

$$p(\boldsymbol{x}_{0:T}) = p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$$

- Conditional

$$p(\boldsymbol{x}_{0:T} \mid y) = p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, y)$$

More controllable!

Conditional Score Matching

- Score matching with conditional information

$$
\begin{aligned}
\nabla \log p\left(\boldsymbol{x}_t \mid y\right) &= \nabla \log \left(\frac{p\left(\boldsymbol{x}_t\right) p\left(y \mid \boldsymbol{x}_t\right)}{p(y)} \right) \\
&= \nabla \log p\left(\boldsymbol{x}_t\right) + \nabla \log p\left(y \mid \boldsymbol{x}_t\right) - \nabla \log p(y) \\
&= \underbrace{\nabla \log p\left(\boldsymbol{x}_t\right)}_{\text{unconditional score}} + \underbrace{\nabla \log p\left(y \mid \boldsymbol{x}_t\right)}_{\text{adversarial gradient}}
\end{aligned}
$$

Classifier Guidance

- Use a discriminative classifier for $\nabla \log p\left(y \mid \boldsymbol{x}_t\right)$

$$\nabla \log p\left(\boldsymbol{x}_t \mid y\right) = \nabla \log p\left(\boldsymbol{x}_t\right) + \gamma \nabla \log p\left(y \mid \boldsymbol{x}_t\right)$$

- γ controls the strength of the condition

- Limitations:
  - Need a separate classifier
    - Trained on noisy images
  - Conditioning depends on the performance of classifier

Classifier-Free Guidance

- Score matching with conditional information

$$\nabla \log p\left(\boldsymbol{x}_t \mid y\right) = \nabla \log p\left(\boldsymbol{x}_t\right) + \gamma \nabla \log p\left(y \mid \boldsymbol{x}_t\right)$$
$$\nabla \log p\left(y \mid \boldsymbol{x}_t\right) = \nabla \log p\left(\boldsymbol{x}_t \mid y\right) - \nabla \log p\left(\boldsymbol{x}_t\right)$$

- Classifier-free guidance

$$\begin{aligned}\nabla \log p\left(\boldsymbol{x}_t \mid y\right) &= \nabla \log p\left(\boldsymbol{x}_t\right) + \gamma\left(\nabla \log p\left(\boldsymbol{x}_t \mid y\right) - \nabla \log p\left(\boldsymbol{x}_t\right)\right) \\ &= \nabla \log p\left(\boldsymbol{x}_t\right) + \gamma \nabla \log p\left(\boldsymbol{x}_t \mid y\right) - \gamma \nabla \log p\left(\boldsymbol{x}_t\right) \\ &= \underbrace{\gamma \nabla \log p\left(\boldsymbol{x}_t \mid y\right)}_{\text{conditional score}} + \underbrace{(1-\gamma) \nabla \log p\left(\boldsymbol{x}_t\right)}_{\text{unconditional score}}\end{aligned}$$

Training of Classifier-Free Guidance

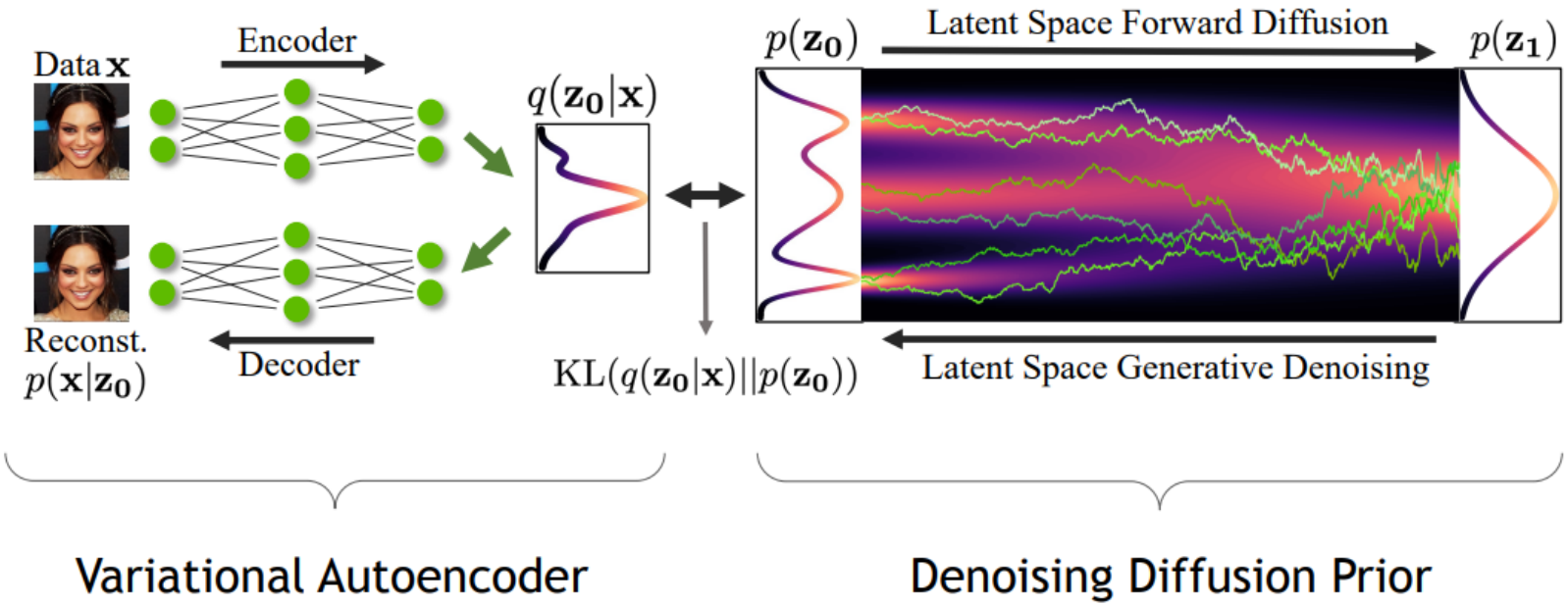
- For conditional embeddings:
  - Randomly drop (with probability p) original conditionals y and replace it with an additional unconditional class

$$\mathbb{E}_{\mathcal{E}(x),y,\epsilon\sim\mathcal{N}(0,1),t}\left[\|\epsilon-\epsilon_\theta\left(z_t,t,\tau_\theta(y)\right)\|_2^2\right]$$

  - By doing so, you train a single network that handles both unconditional and conditional terms

Latent Diffusion Models

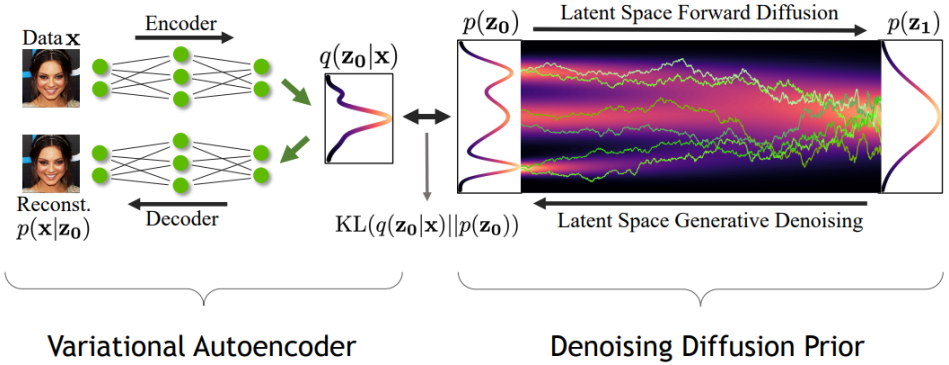
- Core idea
  - Map the input data to an embedding space, on which denoising diffusion models are applied.



Ho et al., "Cascaded Diffusion Models for High Fidelity Image Generation", 2021.
Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", arXiv 2022.
Saharia et al., "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", arXiv 2022.

Latent Diffusion Models

- Core idea
  - Map the input data to an embedding space, on which denoising diffusion models are applied.



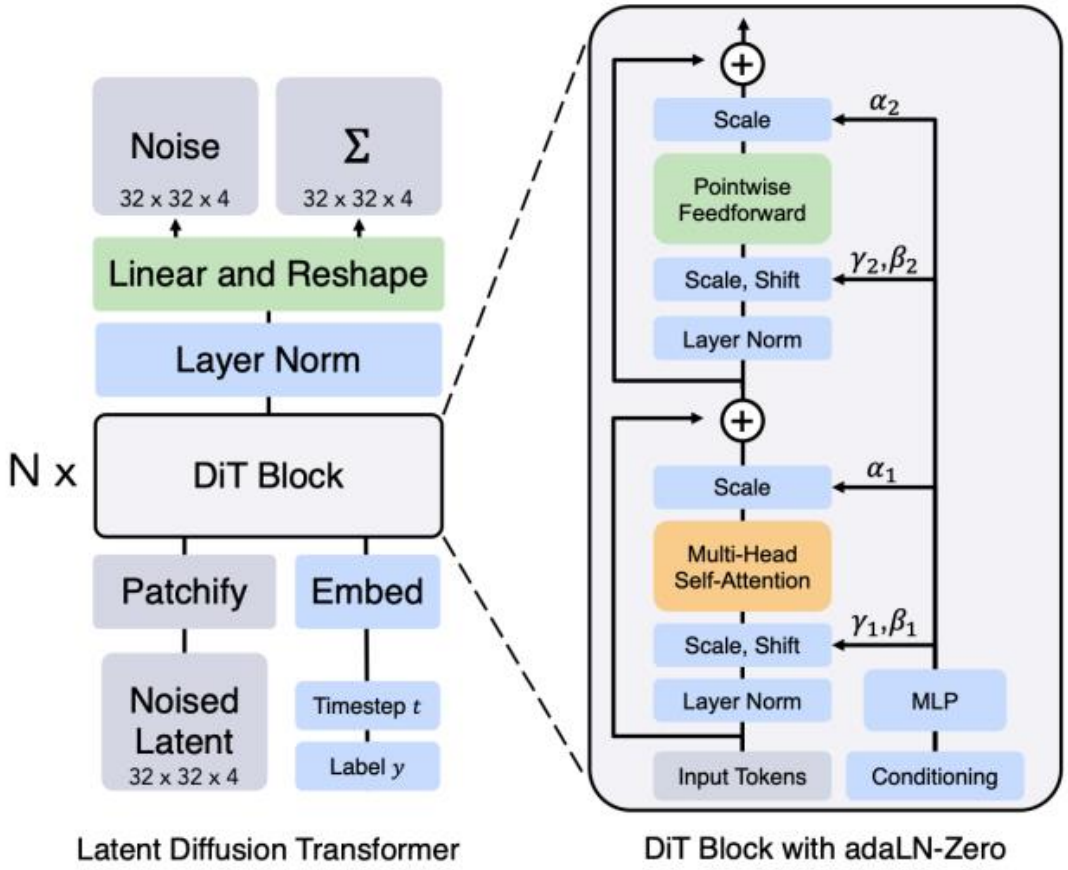
  - Two stage training: train autoencoder first, then train the diffusion prior
  - Focus on compression without any loss in reconstruction quality
  - Demonstrated the expressivity of latent diffusion models on many conditional problems

- The efficiency and expressivity of latent diffusion models + open-source access fueled a large body of work in the community

Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022.
Denoising Diffusion-based Generative Modeling: Foundations and Applications. CVPR2022 Tutorial

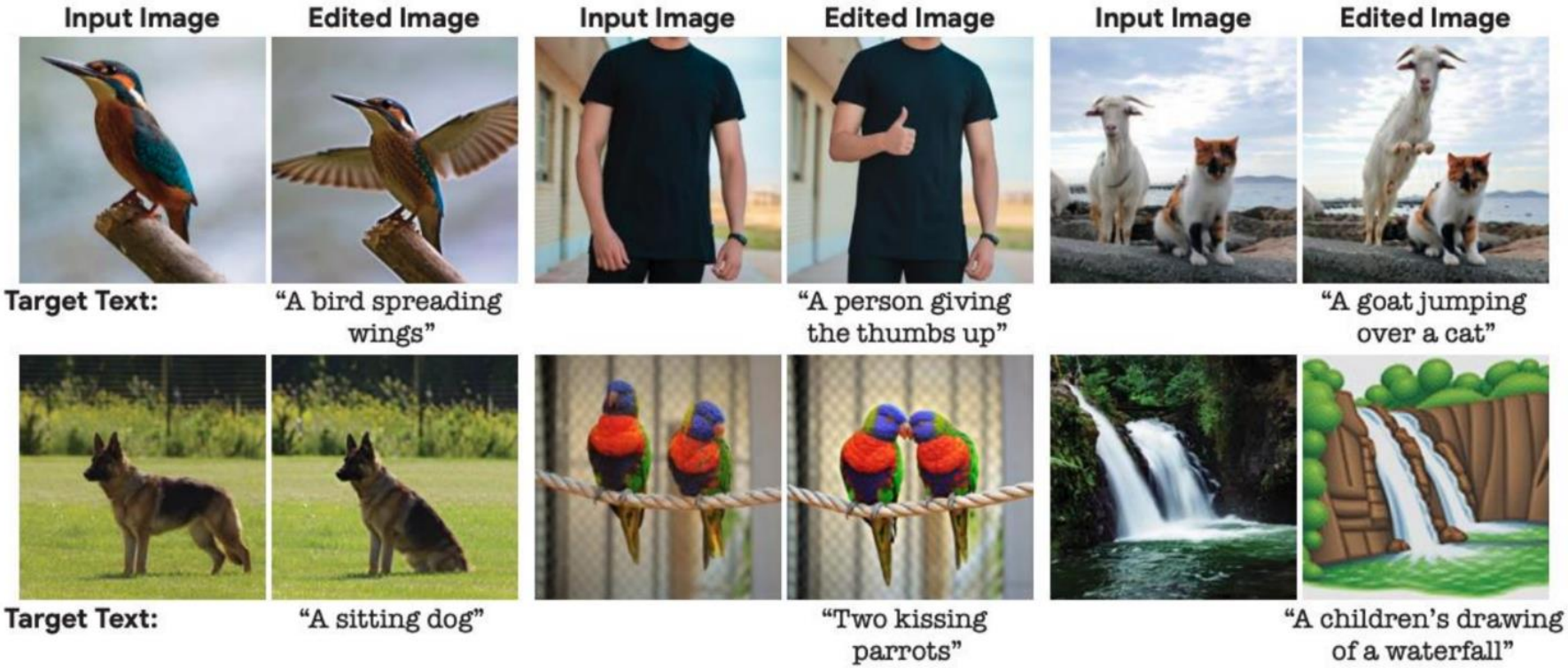
DiT

- A transformer architecture for Diffusion models



Applications

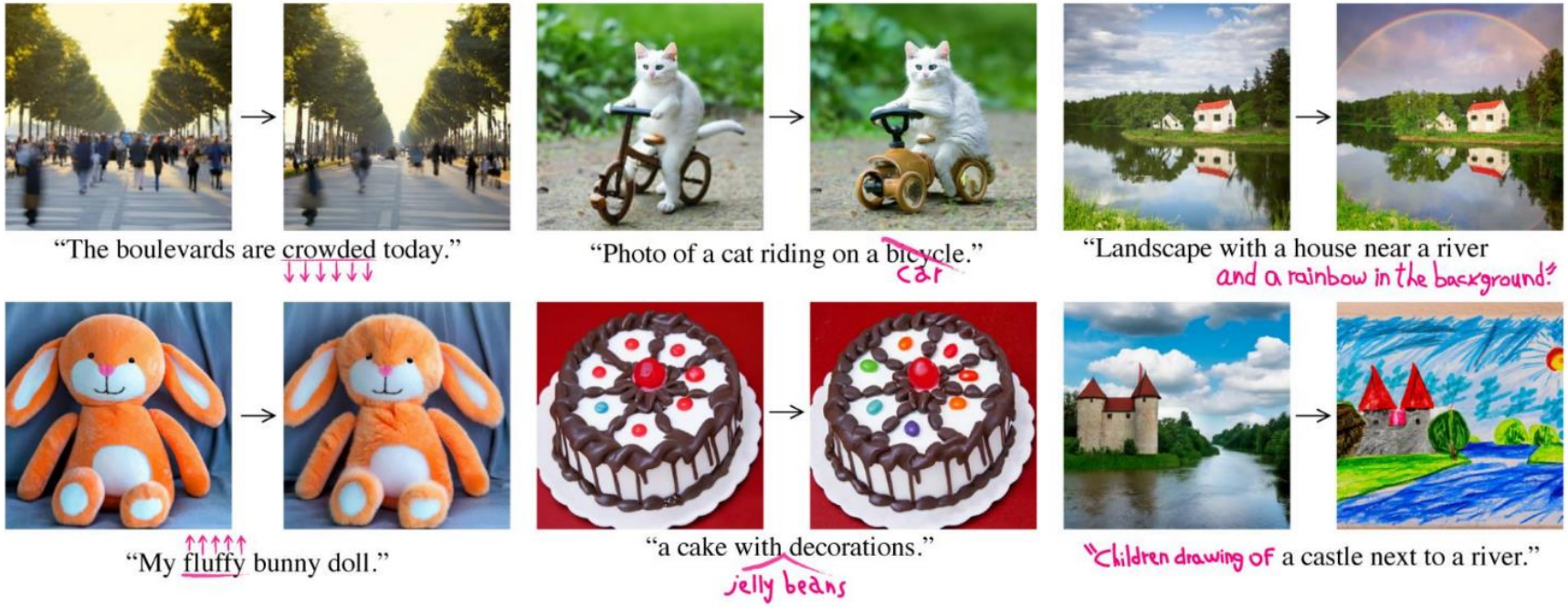
Text-guided image editing



Denoising Diffusion-based Generative Modeling: Foundations and Applications. CVPR2022 Tutorial
Kawar et al., "Imagic: Text-Based Real Image Editing with Diffusion Models", CVPR 2023

Applications

Text-guided image editing



Denoising Diffusion-based Generative Modeling: Foundations and Applications. CVPR2022 Tutorial
Hertz et al., "Prompt-to-Prompt Image Editing with Cross-Attention Control", ICLR 2023

Applications

Text-guided image editing

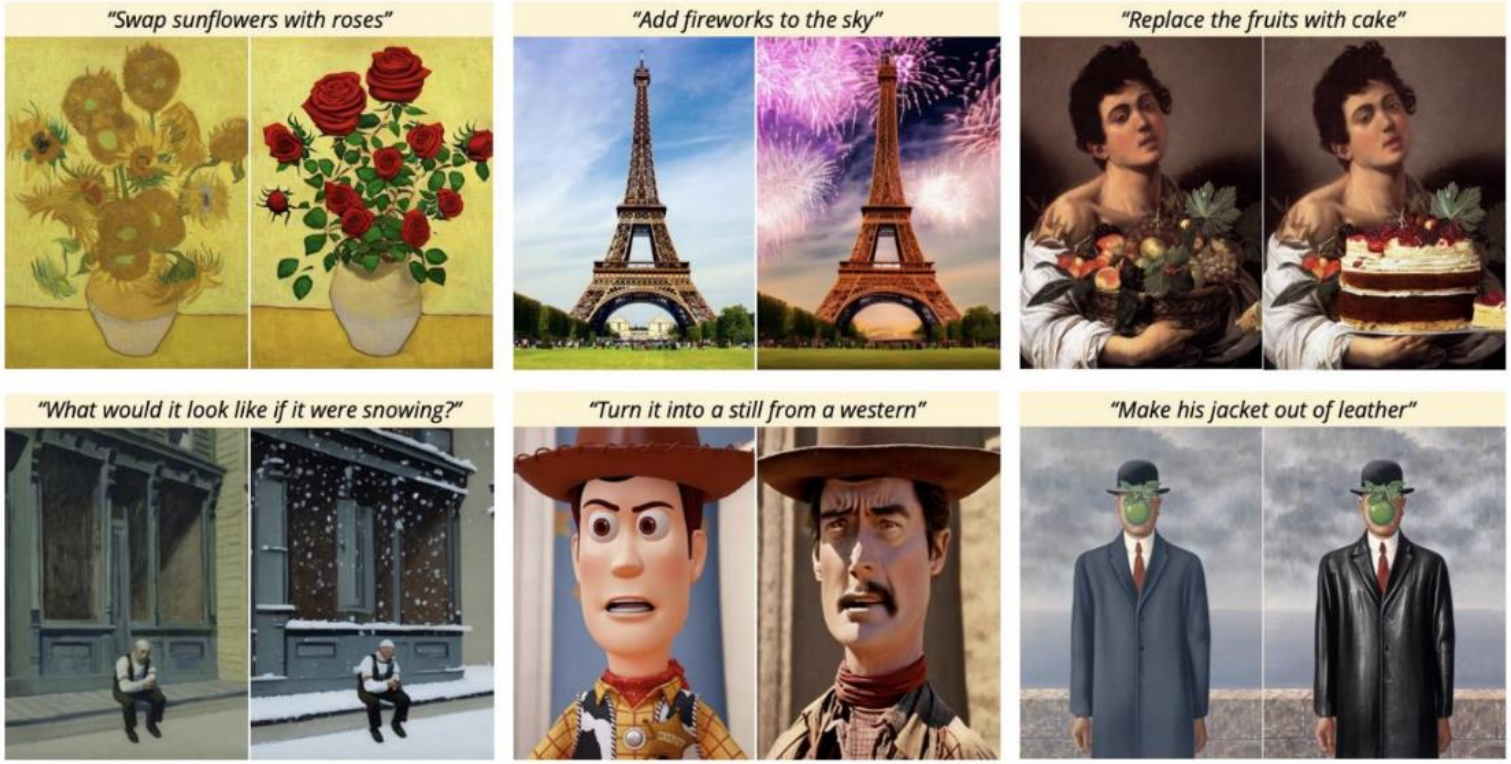


Figure 1. Given **an image** and **an instruction** for how to edit that image, our model performs the appropriate edit. Our model does not require full descriptions for the input or output image, and edits images in the forward pass without per-example inversion or fine-tuning.

Denoising Diffusion-based Generative Modeling: Foundations and Applications. CVPR2022 Tutorial
Brooks et al., "Instructpix2pix: Learning to follow image editing instructions", CVPR 2023

Applications

Personalization with diffusion models

Generated images

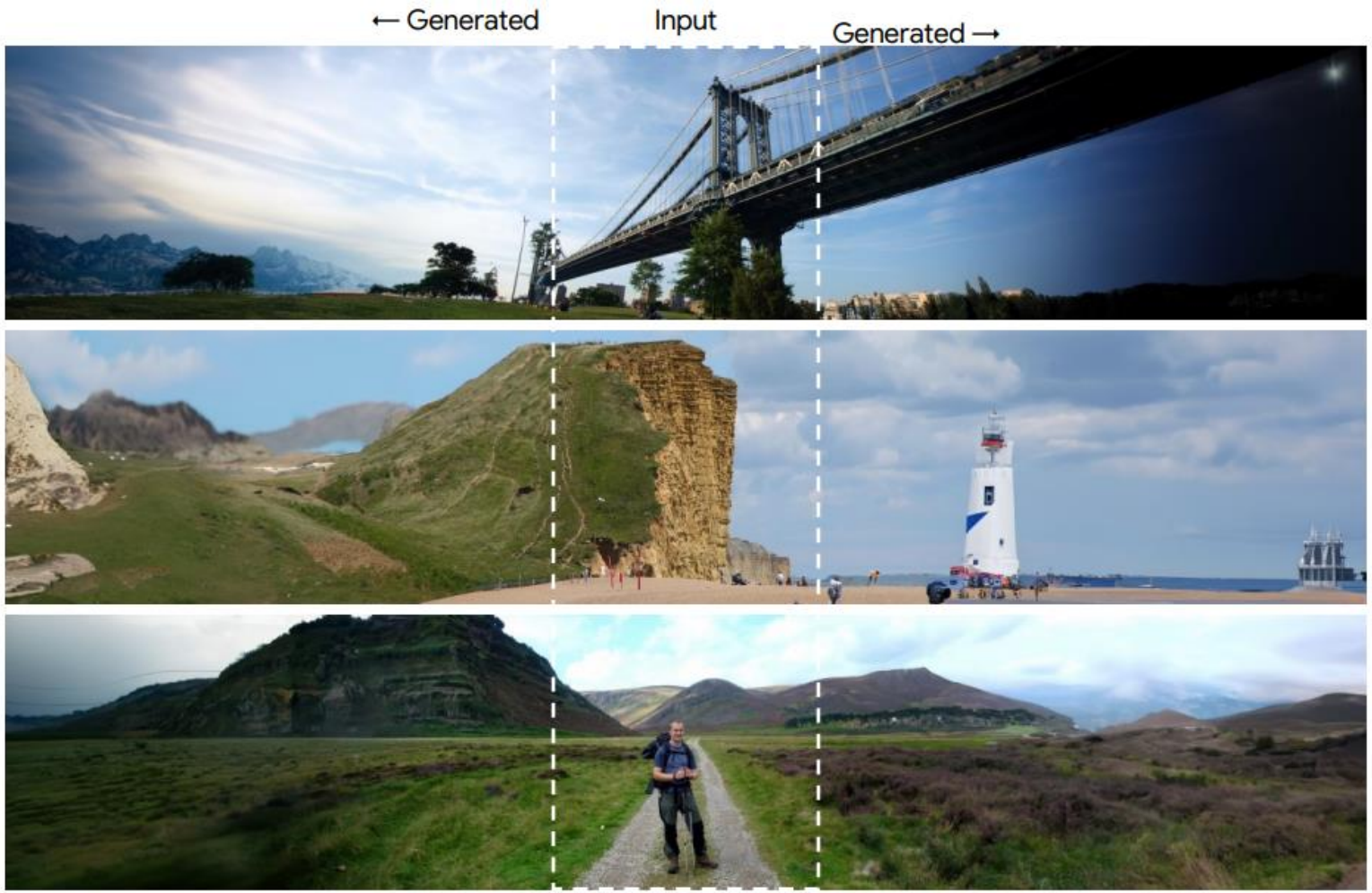
Denoising Diffusion-based Generative Modeling: Foundations and Applications. CVPR2022 Tutorial
Ruiz et al., "DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation", CVPR 2023

Applications

Outfilling/Outpainting

Applications

Outfilling/Outpainting



Palette: Image-to-Image Diffusion Models, 2022